

Orderix / Weper — Documentação de Arquitetura

Gerado a partir do código real dos repositórios `orderix-backend`, `orderix-web` e `orderix-android`.

Data: 30 de agosto de 2026

Este PDF reúne os 18 documentos técnicos da pasta `docs/architecture/`. Nenhum fluxo foi inventado: nomes de classes, endpoints e tabelas correspondem ao código existente.

#	Documento
01	Visão geral do sistema
02	Arquitetura do backend
03	Arquitetura do frontend web
04	Arquitetura do Android
05	Fluxo de autenticação
06	Multi-tenancy
07	Fluxo de pedido
08	Fluxo de mesa
09	Fluxo de comanda
10	Fluxo de balcão
11	Fluxo da cozinha / KDS
12	Fluxo de pagamento
13	Fluxo de estoque
14	Mapa de endpoints
15	Modelo de dados
16	Integrações e dependências
17	Problemas e gaps
18	Fluxo completo do sistema

01 — Visão geral do sistema

Documento gerado a partir do código real dos repositórios `orderix-backend`, `orderix-web` e `orderix-android`. Nada aqui é hipotético: nomes de classes, endpoints e tabelas correspondem ao que existe no código.

O que é o Orderix / Weper

O produto é um sistema de operação para estabelecimentos de alimentação (salão, caixa e cozinha). O display name no backend é **Weper** (`orderix.solutions:orderix:1.0.0`); o Android se chama **Weper Solutions**; o web README chama **Weper Web**.

Não há um quarto cliente. O KDS (cozinha) **não é um aplicativo separado**: é uma rota da SPA web (`/app/cozinha`, componente `Kitchen`).

Stack real

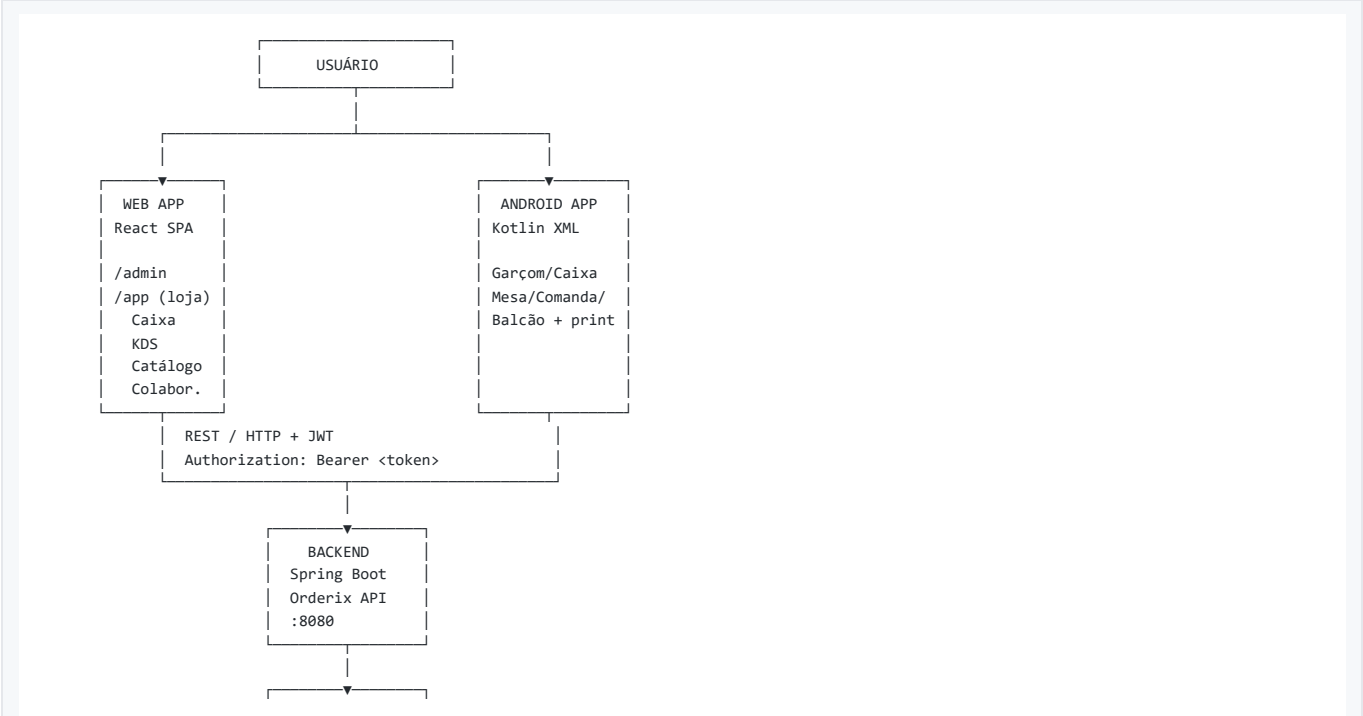
Camada	Tecnologia encontrada
Backend	Spring Boot 3.2.5, Java 17, Maven, JPA/Hibernate, Spring Security, JWT 0.12.6, MapStruct 1.4.2, springdoc-openapi
Banco	PostgreSQL (jdbc:postgresql://.../orderix). Schema via spring.jpa.hibernate.ddl-auto=update. Sem Flyway/Liquibase
Web	React 18.3.1, Create React App (react-scripts 5), JavaScript (sem TypeScript), React Router v6, Axios, MUI 5
Android	Kotlin, XML + ViewBinding (sem Compose), Hilt, Retrofit 2.11, Coroutines + LiveData, DataStore Proto. Sem Room
Realtime	Não existe WebSocket nem SSE. Há polling de 12s no KDS web e no badge de turno do Android
Integrações externas	Nenhuma de negócio (sem gateway de pagamento, e-mail, SMS, S3). Firebase no web está inicializado mas não autentica. Android usa Firebase Crashlytics apenas. Impressora térmica Bluetooth no Android (DantSu ESCPOS)

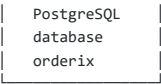
Quem usa o quê

MASTER / SUPER_ADMIN	→ Web /admin	(painel de lojas)
STORE_ADMIN / ADMIN	→ Web /app	(gestão da loja)
CASHIER	→ Web /app/atendimento e Android	(salão/caixa)
KITCHEN	→ Web /app/cozinha	(KDS)
WAITER	→ Android	(pedido, mesa, comanda, balcão)
	→ Web /app/garcom	é apenas placeholder

O Android **rejeita login** se o role não for WAITER OU CASHIER, ou se waiterId for nulo (`AuthFailure.NotWaiter` em `AuthenticationDataSource`).

Arquitetura geral (código real)





```
flowchart TB
    subgraph clientes [Clientes]
        Web["orderix-web<br/>React 18 SPA"]
        Android["orderix-android<br/>Kotlin XML"]
    end

    subgraph backend [orderix-backend]
        API["Spring Boot 3.2.5<br/>JWT + TenantContext"]
    end

    DB["PostgreSQL<br/>tb_*"]

    Web -->|REST Axios<br/>Bearer JWT| API
    Android -->|REST Retrofit<br/>Bearer JWT| API
    API -->|JPA Hibernate<br/>storeFilter| DB
```

Como as três pontas se conectam

- 1. **Web e Android não conversam entre si.** Ambos falam só com o backend.
- 2. O tenant (storeId) **não** vai em header, query ou body nas listagens. Sai do user.store gravado no banco, recarregado a cada request pelo JwtAuthenticationFilter, e entra no TenantContext (ThreadLocal). O Hibernate aplica o filtro storeFilter (store_id = :storeId).
- 3. Pedidos **nascem no Android** (POST /tables/{tableId}/orders). A web **não chama** esse endpoint. O caixa web **consulta e fecha** contas (GET .../orders + POST .../close). O KDS web **lista e muda status** (GET /kitchen/orders + PATCH /kitchen/orders/{id}/status).
- 4. Catálogo, mesas, comandas e colaboradores são cadastrados na **web**. O Android **lê** categorias/produtos/mesas/comandas para montar o pedido.

Constantes de domínio importantes

Constante	Valor	Onde
Mesa/balcão sentinela	999 (OrderService.DEFAULT_ID, COUNTER_TABLE_ID)	Backend e Android. Pedido de balcão e de comanda usa tableId=999
Garçom sentinela de balcão	999 (COUNTER_WAITER_ID)	Android. Backend trata waiterId == 999 como não-mesa
Taxa de serviço	10% (1.10)	Backend GroupOrderService.calculateTotal; web CheckoutDialog também calcula 10% na UI
Polling KDS	12 segundos	Kitchen.js
Polling turno Android	12 segundos	MainActivity.observeShiftBadge → GET /home/shift
JWT validade	app.jwt.expiration-ms default 12h	Sem refresh token

O que o sistema faz de ponta a ponta (real)

```
MASTER cria Store (tb_store + User STORE_ADMIN + Subscription)
↓
STORE_ADMIN cadastra colaboradores, categorias, produtos, mesas, comandas
↓
WAITER/CASHIER no Android faz login → storeId vem do User, sem tela de escolha
↓
Garçom escolhe Mesa OU Comanda OU Balcão → monta carrinho → POST /tables/{id}/orders
↓
Pedido ACTIVE + kitchenStatus NEW → aparece no KDS web (polling 12s)
↓
Cozinha avança NEW → IN_PREPARATION → READY → DELIVERED
↓
Caixa web (ou Android) fecha mesa/comanda/balcão com PaymentMethodEnum
↓
Mesa/comanda volta a isAvailable=true
↓
Dashboard GET /dashboard/overview lê pedidos fechados
```

Diferenças em relação ao fluxo “esperado” do briefing:

- Não existe seleção de estabelecimento no Android: o usuário já pertence a uma Store.

- Não existe baixa de estoque na venda. `Product.quantity` é campo de cadastro; o dashboard só **alerta** quantidade entre 1 e 8.
- Não existe entidade `Payment` / tabela de pagamentos. O método é um enum gravado em `Order` e `GroupOrder` no fechamento.
- KDS não é push: é polling HTTP.
- A web não cria pedidos de atendimento.

Documentos desta pasta

Arquivo	Conteúdo
02-backend-architecture.md	Controllers, services, security, packages
03-web-architecture.md	Rotas, services, contexts, papéis
04-android-architecture.md	Telas, ViewModels, Retrofit, sessão
05-authentication-flow.md	Login JWT ponta a ponta
06-multitenancy-flow.md	<code>storeId</code> , filtro Hibernate, riscos
07-order-flow.md	Nascimento do pedido
08-table-flow.md	Mesa + <code>GroupOrder</code>
09-command-flow.md	Comanda
10-counter-flow.md	Balcão (<code>tableId=999</code>)
11-kds-flow.md	Cozinha / <code>kitchenStatus</code>
12-payment-flow.md	Fechamento e <code>PaymentMethodEnum</code>
13-inventory-flow.md	Estoque (parcial / sem baixa)
14-api-map.md	Tabela de endpoints e origem
15-database-model.md	Entidades, tabelas, cardinalidade
16-integration-map.md	Dependências entre projetos
17-problems-and-gaps.md	Gaps classificados
18-complete-system-flow.md	Documento principal + matriz

02 — Arquitetura do backend

Repositório: orderix-backend. Package raiz: orderix.solutions.backend. Entry point: OrderixApplication.

Stack

- Spring Boot 3.2.5 / Java 17 / Maven
- spring-boot-starter-web, data-jpa, security, aop
- PostgreSQL driver
- JWT 0.12.6
- MapStruct 1.4.2.Final + Lombok
- springdoc-openapi 2.0.2
- Bind: server.address=0.0.0.0, server.port=\${PORT:8080} (compatível com Render)

Packages

```
orderix.solutions.backend
├─ app.home      HomeController, ShiftOverviewController
├─ auth          AuthController, AuthService, JwtTokenProvider
├─ category      Category CRUD
├─ comanda       Comanda CRUD
├─ dashboard     Dashboard operacional da loja
├─ employee      Funcionários (User da loja)
├─ exception     GlobalExceptionHandler, ApiError
├─ media         DefaultMenuImage
├─ order         Order, GroupOrder, Kitchen, ComandaAccount, Counter
├─ product       Product, Extra, MandatoryGroup
├─ security      SecurityConfig, JWT filter, TenantContext, CORS, Hibernate aspect
├─ store         Store + StoreAccess + StoreFilters
├─ subscription  planos/assinatura
├─ table         mesas
├─ user          User, Role, UserPrincipal
├─ waiter        garçons (entidade Waiter 1:1 User)
├─ DataInitializer bootstrap SUPER_ADMIN / demo seed
├─ GroupOrderMigration ApplicationRunner legado
```

Controllers

Controller	@RequestMapping	Papel
AuthController	/api/auth	Login e /me
StoreController	/api/stores	CRUD lojas (MASTER/SUPER_ADMIN)
EmployeeController	/api/employees	CRUD usuários da loja
WaiterController	/api/waiters	CRUD entidade Waiter
CategoryController	/category	CRUD categorias
ProductController	/product	CRUD produtos
ProductExtraController	/product/extra	Extras
MandatoryGroupController	/product/mandatory-groups	Grupos obrigatórios
TableController	/tables	CRUD mesas
GroupOrderController	/tables	Pedidos de mesa/balcão
ComandaController	/comandas	CRUD comandas
ComandaOrderController	/comandas	Conta da comanda
CounterOrderController	/counter/orders	Fechar pedido de balcão
OrderController	/order	GET /all ativo; new/edit/getByTable deprecated
KitchenController	/kitchen	KDS
DashboardController	/dashboard	Overview da loja
HomeController	/home	Cards estáticos Balcão/Mesas/Comandas
ShiftOverviewController	/home	Overview do turno do garçom

Lista completa de métodos: [14-api-map.md](#).

Services

Service	Responsabilidade
AuthService	Login (email ou username), BCrypt, loja ativa, JWT, alignWaiterStore
StoreService	CRUD Store, cria User STORE_ADMIN + Subscription
SubscriptionService	Criar/atualizar assinatura; isSubscriptionActive
EmployeeService	CRUD User da loja; roles STORE_ADMIN/WAITER/KITCHEN/CASHIER; ensureForUser se floor operator
WaiterService	CRUD Waiter + User; ensureForUser
CategoryService	CRUD categorias
ProductService	CRUD produtos; campo quantity é cadastro, não movimento
ProductExtraService	Extras
MandatoryGroupService	Grupos obrigatórios e assign a produto
TableService	CRUD mesas (deleted)
ComandaService	CRUD comandas
OrderService	Cria Order + OrderItem + extras + mandatory; fecha balcão; totais
OrderMandatoryValidationService	Valida seleções obrigatórias
GroupOrderService	Conta de mesa; tableId==999 desvia para pedido avulso
ComandaAccountService	Conta aberta e fechamento de comanda
KitchenService	Lista KDS + atualiza KitchenStatus
DashboardService	Overview financeiro/operacional/estoque crítico
ShiftOverviewService	Visão do turno

Repositories ↔ entidades

Repository	Entidade	Tabela
StoreRepository	Store	tb_store
UserRepository	User	tb_user
SubscriptionRepository	Subscription	tb_subscription
CategoryRepository	Category	tb_category
ProductRepository	Product	tb_product
ProductExtraRepository	ProductExtra	tb_product_extra
MandatoryGroupRepository	MandatoryGroup	tb_mandatory_group
MandatoryItemRepository	MandatoryItem	tb_mandatory_item
TableRepository	Table	tb_table
ComandaRepository	Comanda	tb_comanda
WaiterRepository	Waiter	tb_waiter
OrderRepository	Order	tb_order
OrderItemRepository	OrderItem	tb_order_item
OrderProductExtraRepository	OrderProductExtra	tb_order_product_extra
OrderMandatorySelectionRepository	OrderMandatorySelection	tb_order_mandatory_selection
GroupOrderRepository	GroupOrder	tb_group_order

DTOs e mappers

MapStruct (@Mapper(componentModel = "spring")): OrderMapper, ProductMapper, CategoryMapper, TableMapper, ComandaMapper, WaiterMapper, MandatoryGroupMapper, MandatoryItemMapper.

Montagem manual: Store, Employee, Auth, GroupOrder, Dashboard, Shift.

DTOs de request/response relevantes: LoginRequest/LoginResponse, AuthUserDTO, CreateStoreRequest, UpdateStoreRequest, CreateEmployeeRequest, EmployeeDTO, WaiterDTO, CategoryDTO, ProductDTO, ProductExtraDTO, MandatoryGroupDTO, OrderDTO, GroupOrderDTO, CloseGroupOrderRequest, CloseCounterOrderRequest, KitchenStatusRequest, StoreOverviewDTO, ShiftOverviewDTO, HomeDTO.

Segurança

Não há @PreAuthorize. Tudo em SecurityConfig.authorizeHttpRequests.

Público: OPTIONS /**, Swagger, GET /images/**, POST /api/auth/login.

Recurso	Roles
/api/stores/**	MASTER, SUPER_ADMIN
Mutação waiters/employees	ADMIN, STORE_ADMIN, MASTER, SUPER_ADMIN
Mutação tables/comandas/category/product	idem
Fechar mesa/comanda/balcão	WAITER, CASHIER + admins
PATCH kitchen status	WAITER, CASHIER, KITCHEN + admins
GET /kitchen/**	KITCHEN + admins (WAITER não lista o KDS)
Demais	autenticado

UserPrincipal.getAuthorities(): MASTER e SUPER_ADMIN compartilham ambos os roles; ADMIN e STORE_ADMIN também.

Senha: BCrypt (PasswordEncoder bean). JWT HMAC, claims sub (userId), username, role, storeId (se houver loja), iat, exp.

Em runtime o storeId efetivo **não** vem do claim: JwtAuthenticationFilter recarrega User via findByIdWithStore e seta TenantContext.

Assinatura inativa: filtro responde **403** SUBSCRIPTION_INACTIVE (exceto /api/auth*).

Multi-tenancy

Ver [06-multitenancy-flow.md](#). Resumo: TenantContext + Hibernate @Filter storeFilter via StoreHibernateFilterAspect em *Service.* (exceto auth, StoreService, SubscriptionService, WaiterService.ensureForUser). StoreAccess.owned(...) valida entity.store.id == TenantContext.storeId.

Jobs / realtime / integrações

- Sem @Scheduled, @Async, WebSocket, SSE, filas.
- DataInitializer (ApplicationRunner): SUPER_ADMIN, patches de schema, seed demo opcional (DEMO_SEED).
- GroupOrderMigration (ApplicationRunner): migração legada de group orders.
- Sem gateway de pagamento, e-mail, storage de arquivos (imagens de cardápio vão como TEXT/base64 no JSON).

Banco

- URL: \${DB_URL} default jdbc:postgresql://localhost:5432/orderix
- ddl-auto=\${DDL_OPTION:update}
- Sem migrations versionadas
- CORS: \${CORS_ALLOWED_ORIGINS:*}

03 — Arquitetura do frontend web

Repositório: orderix-web. SPA React 18 + CRA. Linguagem JavaScript.

Base HTTP: process.env.REACT_APP_API_BASE_URL || 'http://localhost:8080' (src/services/apiConfig.js).

Estrutura

```
src/
├─ App.js                rotas raiz
├─ services/             apiConfig, auth, session, accessControl, firebase
├─ theme/                ThemeContext
├─ commons/              hooks/componentes compartilhados
├─ routes/
│  ├─ ProtectedRoute.js
│  ├─ home/              landing pública
│  ├─ login/
│  ├─ privacy/, subscription/
│  ├─ master/            /admin (MASTER / SUPER_ADMIN)
│  └─ dashboard/         /app (loja)
│     ├─ atendimento/, comandas/, tables/, kitchen/
│     ├─ menu/ (category, product)
│     ├─ employees/, dashboard-gerencial/
│     ├─ inventory/, settings/, financeiro / mock
│     └─ WaiterHome.js
```

Rotas

Definidas em App.js. Paths canônicos em accessControl.js (PATHS).

Path	Componente	Guard
/	Home	público
/login	Login	público
/forgot-password	ForgotPassword	público, sem API
/privacy, /terms	políticas	público
/subscription-blocked	SubscriptionBlocked	ProtectedRoute
/app, /app/:section	Dashboard	StoreLayout + ProtectedRoute
/admin, /admin/:section	MasterDashboard	AdminLayout + ProtectedRoute
/dashboard	redirect /app	legado
/master/dashboard	redirect /admin	legado

Seções /app/:section (Dashboard.js + listItems.js)

Path	Componente	Quem acessa
/app/dashboard	DashboardGerencial	ADMIN / STORE_ADMIN
/app/atendimento	Atendimento	ADMIN / STORE_ADMIN / CASHIER
/app/cozinha	Kitchen	ADMIN / STORE_ADMIN / KITCHEN
/app/produtos	Menu	ADMIN / STORE_ADMIN
/app/colaboradores	Employees	ADMIN / STORE_ADMIN
/app/garcom	WaiterHome	WAITER — placeholder , sem HTTP

Paths em PATHS **sem tela montada**: /app/estoque, /app/configuracoes.

Seções /admin/:section

Path	Componente
/admin/dashboard	MasterOverview
/admin/restaurantes	lista + form inline no MasterDashboard

/admin/planos	MasterPlans
/admin/assinaturas	MasterSubscriptions

Autenticação e sessão

Item	Implementação
Login	authService.login → POST /api/auth/login { email, password }
Token	localStorage chave orderix.token (session.js)
User	localStorage chave orderix.user
Refresh	não existe
Interceptor request	Authorization: Bearer \${token}
Interceptor 401	clearSession + redirect /login
Interceptor 403 com "assinatura"	marca subscriptionActive: false → /subscription-blocked
Firebase	initializeFirebase() em App.js; não usado no login
Google OAuth	botão desabilitado (GOOGLE_OAUTH_ENABLED)

Roles

ROLES em accessControl.js: MASTER, SUPER_ADMIN, STORE_ADMIN, ADMIN, WAITER, KITCHEN, CASHIER.

ProtectedRoute exige autenticação + canAccessRoute (assinatura + storeId para não-platform + role). Menu filtrado por dashboardItems[].roles.

ROLE_PERMISSIONS é stub: o RBAC fino da UI é só rota/menu; a autorização real está no JWT/backend.

Estado global

Sem Redux/Zustand. Contexts:

Context	Arquivo	Papel
AppThemeProvider	theme/ThemeContext.js	tema
ProductProvider	menu/product/providers/ProductContext.js	UI produtos
CategoryProvider	menu/category/providers/CategoryContext.js	UI categorias
TablesProvider	tables/provider/TablesContext.js	UI mesas
ComandasProvider	comandas/provider/ComandasContext.js	UI comandas
EmployeesProvider	employees/provider/EmployeesContext.js	UI colaboradores

Services HTTP (todos)

Service	Endpoints
authService	POST /api/auth/login
storesService	GET/POST /api/stores, GET/PUT /api/stores/{id}, GET /api/stores/dashboard
employeesService	CRUD /api/employees
categoryService	/category/all, /new, /update, /deleteById
productService	/product/all, /new, /update, /delete, /getByCategoryId
tablesService	/tables/all, /new, /delete
comandasService	/comandas/all, /new, /delete
accountService	GET /tables/{id}/orders, POST /tables/{id}/group-orders/close, equivalentes de comanda
kitchenService	GET /kitchen/orders, PATCH /kitchen/orders/{id}/status
dashboardService	GET /dashboard/overview?period=

A web **não** chama POST /tables/{id}/orders (criação de pedido), /counter/orders/{id}/close, /home/*, /api/waiters, /product/extra, /api/auth/me.

Mapa tela → API

```
LoginPage / LoginForm
→ authService.login
→ POST /api/auth/login

MasterDashboard (Restaurantes)
→ storesService
→ GET/POST /api/stores, PUT /api/stores/{id}, GET /api/stores/dashboard

Employees
→ employeesService
→ CRUD /api/employees

Menu (Category + Product)
→ categoryService + productService
→ /category/* e /product/*

Atendimento (Tables + Comandas + CheckoutDialog)
→ tablesService, comandasService, accountService
→ CRUD mesas/comandas + GET conta + POST close

Kitchen
→ kitchenService
→ GET /kitchen/orders (polling 12s)
→ PATCH /kitchen/orders/{id}/status

DashboardGerencial
→ dashboardService
→ GET /dashboard/overview

WaiterHome, ForgotPassword, Settings, Inventory
→ sem API de negócio
```

Tenant na web

Comentário em `accessControl.js`: tenant vem do JWT; query params **não** devem trocar de loja. Requests enviam só `Authorization: user.storeId` fica na sessão para UI (`getRestaurantId`).

Realtime

Polling 12s em `Kitchen.js` (`setInterval(loadOrders, 12000)`). Atendimento e dashboard recarregam sob ação do usuário.

Telas órfãs / mock

- `Inventory.js`, `Settings.js` — mock local, não montados no Dashboard
- `Financial`, `Orders`, `Overview`, `Chart`, `Deposits`, `Establishment`, `Checkout.js` — legado não ligado às rotas atuais
- `MasterOverview` usa `mockRevenueHistory.js` para parte do gráfico de receita
- `getStoreById` existe no service e **não é chamado**

04 — Arquitetura do Android

Repositório: orderix-android. Módulo único :app. Package: orderix.solutions. Application: OrderixApplication (@HiltAndroidApp).

Stack real (vs README)

O README cita Room, Firebase Auth, Firestore e Picasso. O Gradle/código **não** têm Compose runtime de UI, Room, Koin, WorkManager nem FCM messaging.

Item	Realidade
UI	XML + ViewBinding, Navigation + Safe Args
DI	Hilt 2.51.1
Rede	Retrofit 2.11 + Gson + OkHttp
Async	Coroutines + LiveData (StateFlow só em ShiftAttentionStore)
Sessão	DataStore Proto (user_session.pb)
Offline	Não (sem Room; cache em memória de produtos)
Impressão	ESCPOS ThermalPrinter Bluetooth
Crash	Firebase Crashlytics
minSdk / target	26 / 35

BuildConfig.BASE_URL: debug via -PdebugApiUrl OU local.properties debug.api.url (ex.: http://192.168.15.9:8080/); release placeholder https://YOUR-BACKEND-HOST.onrender.com/.

Activities (Manifest)

Activity	Papel
LoginActivity	LAUNCHER
MainActivity	Shell + bottom nav + polling de turno
ForgotPasswordActivity	Stub local, sem API
ThermalPrinterActivity	Bluetooth

RegisterActivity existe e **não** está no Manifest.

Fragments (nav_graph.xml)

Fragment	Função
HomeFragment	Cards Balcão / Mesas / Comandas + turno
AtendimentoFragment	Hub de atendimento
TablesFragment	Lista mesas
ComandasFragment	Lista comandas
CategoriesFragment	Catálogo
ProductsFragment	Produtos da categoria
ProductDetailFragment	Detalhe, extras, mandatory, qty
CartFragment	Carrinho
OrderConfirmationFragment	Confirmação (OrderConfirmationViewModel vazio)
GroupOrderFragment	Conta da mesa/comanda
CounterOrderFragment	No graph, sem action de entrada a partir do Home (balcão vai a categorias)
OrdersFragment / OrderDetailsFragment	Lista/detalhe
ShiftOrdersFragment	Pedidos do turno (prontos/preparo)
AccountFragment	Conta / logout / tema

Camadas por feature

Padrão: presentation (Fragment + ViewModel) → domain (UseCase + Repository interface) → data (RepositoryImpl + DataSource + Retrofit API).

Carrinho **não** tem repository: estado só em CartViewModel.

ViewModels e UseCases

ViewModel	UseCases / repos principais
LoginViewModel	PostAuthenticationUseCase
HomeViewModel	GetHomeOptionsUseCase, GetShiftOverviewUseCase
TablesViewModel	GetAllTablesUseCase, GetGroupOrderByTableUseCase
ComandasViewModel	GetAllComandasUseCase, GetGroupOrderByComandaUseCase
CategoriesViewModel	GetCategoriesUseCase
ProductsViewModel / ProductDetailViewModel	GetProductsUseCase + extras
CartViewModel	CreateOrderInGroupUseCase
GroupOrderViewModel	close mesa/comanda, UpdateKitchenStatusUseCase
OrderDetailsViewModel	CloseCounterOrderUseCase
OrdersViewModel	GetAllOrdersUseCase (GET /order/all)
ThermalPrinterViewModel	scan/connect/print
ForgotPasswordViewModel / RegisterViewModel	stubs

Retrofit APIs

Interface	Endpoints
AuthAPI	POST api/auth/login
HomeAPI	GET home/options/all, GET home/shift
TablesAPI	GET tables/all, GET tables/{tableId}/orders
ComandasAPI	GET comandas/all, GET comandas/{id}/orders
CategoriesAPI	GET category/all
ProductsAPI	GET product/all, GET product/getByCategoryId, POST product/new, extras
GroupOrderAPI	POST tables/{id}/orders, close mesa/comanda/balcão
OrderAPI	POST order/new (wired no DI; carrinho não usa), GET order/all
KitchenAPI	PATCH kitchen/orders/{id}/status

Autenticação

```
LoginActivity
→ LoginViewModel
→ PostAuthenticationUseCase
→ AuthenticationRepository
→ AuthAPI
→ POST api/auth/login
```

Após resposta:

- 1. Aceita só WAITER OU CASHIER e waiterId != null.
- 2. Grava UserSession no DataStore Proto (token, storeId, storeName, waiterId, role...).
- 3. AuthInterceptor envia Authorization: Bearer.
- 4. RefreshTokenInterceptor: em 401 **limpa sessão** e abre LoginActivity. **Não renova token** (há campo refreshToken praticamente não usado).

Não há tela de seleção de estabelecimento. storeId vem no AuthUserDto do login e não é reenviado nas requests.

Fluxos de tela → API

```
LOGIN
LoginActivity → AuthAPI → POST api/auth/login
```

```

HOME
HomeFragment → HomeAPI → GET home/options/all
                → GET home/shift    (também a cada 12s na MainActivity)

MESA
TablesFragment → TablesAPI GET tables/all
                → GroupOrderFragment → GET tables/{tableId}/orders
                → Categories..Cart → POST tables/{tableId}/orders
                → fechar → POST tables/{tableId}/group-orders/close

COMANDA
ComandasFragment → GET comandas/all
                  → setComandaInfo (tableId forçado 999)
                  → POST tables/999/orders (body com comandaId)
                  → GET comandas/{id}/orders
                  → POST comandas/{id}/orders/close

BALCÃO
HomeFragment.startCounterSale() (tableId=999, waiterId=999)
  → catálogo → carrinho (exige paymentType)
  → POST tables/999/orders (paymentMethod no body)
  → POST counter/orders/{orderId}/close (se fechar depois)

KDS
Não há tela KDS. Garçom pode PATCH kitchen/orders/{id}/status
e ver prontos via GET home/shift.

```

Offline / sync

Inexistente para domínio. UI de “Sincronização” na conta é texto estático. Marks locais do garçom (WaiterOrderMarksRepository, Preferences DataStore) **não** fecham conta no servidor.

Gaps Android

- OrderAPI.createOrderWithProducts (POST /order/new) está no DI, mas o backend **lança deprecated** e o cart usa GroupOrderAPI.
- OrdersViewModel força display (saleType="balcao", isPaid=true, orderStatus="fazendo").
- CounterOrderFragment no graph sem navegação clara do Home.
- Pastas vazias: checkout, tableSales, contact.
- Role KITCHEN aparece em label da conta, mas o login rejeita quem não é WAITER/CASHIER.

05 — Fluxo de autenticação

Fluxo esperado vs real

O briefing previa: login → validar usuário → validar estabelecimento → validar permissões → JWT → armazenar → redirecionar.

O código **corresponde em grande parte**, com diferenças:

- Permissões de tela **não** são validadas no AuthService. O backend autentica qualquer user ativo com senha correta e loja ativa. O **cliente** decide o que fazer com o role (web: getPostLoginPath; Android: rejeita se não for WAITER/CASHIER com waiterId).
- Não há refresh token. Expiração default 12h (JWT_EXPIRATION_MS).
- GET /api/auth/me existe no backend; **nenhum cliente o chama** no fluxo normal.

Diagrama

```
sequenceDiagram
    participant U as Usuário
    participant C as Web ou Android
    participant AC as AuthController
    participant AS as AuthService
    participant UR as UserRepository
    participant JWT as JwtTokenProvider
    participant DB as PostgreSQL tb_user / tb_store

    U->>C: email + senha
    C->>AC: POST /api/auth/login
    AC->>AS: login(LoginRequest)
    AS->>UR: findByEmailIgnoreCase ou findByUsernameIgnoreCase
    UR->>DB: SELECT
    AS->>AS: active == true
    AS->>AS: BCrypt matches passwordHash
    AS->>AS: se não platform admin e store != null: store.active
    AS->>AS: TenantContext.set + alignWaiterStore (WAITER/CASHIER)
    AS->>JWT: generateToken(user)
    JWT-->>AS: JWT HS256
    AS-->>C: LoginResponse token + AuthUserDTO
    C->>C: persiste token e user
    C->>C: redireciona por role
```

Backend — passo a passo

Arquivos: AuthController, AuthService, JwtTokenProvider, JwtAuthenticationFilter.

1. POST /api/auth/login (público). Body LoginRequest: email **ou** username + password.
2. AuthService.login:
 - Credenciais vazias → AUTH_INVALID_CREDENTIALS
 - Usuário inexistente / inativo / senha errada → mesma mensagem (não distingue)
 - Se user.store != null e role não é MASTER/SUPER_ADMIN e store.active != true → STORE_INACTIVE
3. TenantContext.set(userId, storeId, role) só durante o login para alignWaiterStore:
 - Se role é floor operator (WAITER OU CASHIER), WaiterService.ensureForUser garante waiter
 - Se waiter.store ≠ User.store, o User é atualizado para a loja do Waiter
4. JWT claims: sub=userId, username, role, storeId (omitido se user.store == null), iat, exp.
5. Response LoginResponse: token, tokenType=Bearer, expiresIn, user (AuthUserDTO: id, name, username, email, role, storeId, storeName, waiterId, subscription*).

Request autenticada

```
Authorization: Bearer <jwt>
↓
JwtAuthenticationFilter
↓
JwtTokenProvider.parse (assinatura + exp)
↓
UserRepository.findByIdWithStore(sub)
↓
user.active?
↓
UserPrincipal + SecurityContext
```

```

↓
TenantContext.set(id, storeId do User no DB, role)
↓
se não platform admin e assinatura inativa → 403 SUBSCRIPTION_INACTIVE
↓
SecurityConfig.authorizeHttpRequests (roles)

```

O claim `storeId` do token **não** é usado para o tenant em runtime. Se o User mudar de loja no banco, o próximo request já usa a loja nova, mesmo com token antigo (até expirar).

Web

```

LoginForm.signIn
→ authService.login(email, password)
→ POST {API_BASE_URL}/api/auth/login
→ saveSession(token, user) // localStorage orderix.token / orderix.user
→ resolveHomePath(user) // accessControl.getPostLoginPath

```

Destinos:

Role	Home
MASTER / SUPER_ADMIN	/admin/dashboard
ADMIN / STORE_ADMIN	/app/dashboard
CASHIER	/app/atendimento
KITCHEN	/app/cozinha
WAITER	/app/garcom (placeholder)

Logout: `clearSession()`. 401 no interceptor: mesma coisa + `/login`.

Android

```

LoginActivity
→ LoginViewModel
→ PostAuthenticationUseCase
→ AuthenticationRepository / AuthAPI
→ POST api/auth/login
→ AuthenticationDataSource:
    role WAITER ou CASHIER E waiterId != null
    senão AuthFailure.NotWaiter
→ SessionRepository.saveUserSession (DataStore Proto)
→ MainActivity

```

Não há escolha de loja. `storeName` só aparece na toolbar.

Autorização (não é login, mas acoplada)

Ver tabela em [02-backend-architecture.md](#). Pontos que divergem da intuição:

- WAITER **pode** PATCH status de cozinha, mas **não** GET `/kitchen/**` (SecurityConfig: GET kitchen só KITCHEN + admins).
- WAITER/CASHIER **podem** fechar mesa/comanda/balcão no backend; a web de caixa também.

O que não existe

- Refresh token / sliding session
- OAuth/Google (UI morta no web)
- Forgot password API (web e Android são stubs)
- Registro self-service (Android `RegisterActivity` fora do Manifest; web não tem)
- 2FA
- GET `/api/auth/me` no fluxo dos clientes

06 — Multi-tenancy

O identificador de tenant no código é `storeId` (`store_id` nas tabelas). Não existem `tenantId`, `establishmentId` nem `restaurantId` no backend. Na web, `getRestaurantId(user)` é um alias de `user.storeId`.

Como o `storeId` é definido

```
User.store (ManyToOne em tb_user.store_id)
↓
Login: AuthUserDTO.storeId + JWT claim storeId (informativo)
↓
Cada request: JwtAuthenticationFilter recarrega User.findByIdWithStore
↓
TenantContext.set(userId, principal.getStoreId(), role)
↓
StoreHibernateFilterAspect ativa storeFilter nas *Service
↓
SQL: store_id = :storeId
```

Não chega por:

- header (X-Store-Id etc.)
- path `/stores/{id}/products`
- query `?storeId=`
- body nas operações da loja (o service faz `TenantContext.storeRef()` ao persistir)

MASTER/SUPER_ADMIN tipicamente têm `User.store == null` → `storeId` nulo → filtro Hibernate **não** é ligado nesses services de loja (early return no aspect se role é platform admin).

Camadas de isolamento

1. **JWT + User no banco** — o usuário pertence a no máximo uma loja.
2. **TenantContext** — ThreadLocal por request; `clear()` no `finally` do filtro.
3. **StoreHibernateFilterAspect** — `@Around` em `*Service.*` exceto:
 - package auth
 - StoreService
 - SubscriptionService
 - WaiterService.ensureForUser
4. **@Filter(name = "storeFilter")** nas entidades com `store_id`.
5. **StoreAccess.owned** — se `TenantContext.getStoreId()` != null, exige `entity.store.id` igual; senão **retorna sem checar**.
6. **Queries explícitas** `existsByIdAndStore_Id` em OrderService (produto, categoria, mesa, waiter, comanda).

Entidades com filtro Hibernate

Entidade	Filtro
Category, Product, ProductExtra, MandatoryGroup	sim
Table, Comanda, Waiter, Order, GroupOrder	sim
Store	só @FilterDef (definição), não se filtra a si
User	não tem @Filter
Subscription	não
MandatoryItem, OrderItem, OrderProductExtra, OrderMandatorySelection	não (filhos; sem store_id)

Cadastro de loja (origem do tenant)

Só MASTER/SUPER_ADMIN, POST `/api/stores`.

```
MasterDashboard (web /admin/restaurantes)
→ storesService.createStore
→ POST /api/stores
→ StoreController
→ StoreService.createStore
→ tb_store (active=true, slug único)
```



```
→ tb_user (role STORE_ADMIN, store_id da loja nova)
→ SubscriptionService.createForStore → tb_subscription
```

O admin da loja passa a ter storeId no próximo login. Colaboradores criados depois herdam TenantContext.storeRef().

Diagrama

```
flowchart TB
    subgraph lojaA [Loja A storeId=1]
        UA[Users]
        PA[Products]
        OA[Orders]
    end

    subgraph lojaB [Loja B storeId=2]
        UB[Users]
        PB[Products]
        OB[Orders]
    end

    JWT[JWT Bearer]
    TC[TenantContext ThreadLocal]
    HF[Hibernate storeFilter]

    JWT --> TC
    TC -->|"store_id = 1"| HF
    HF --> lojaA
    HF -. ->|"não retorna"| lojaB
```

Riscos encontrados no código

CRÍTICO — MASTER/SUPER_ADMIN sem store enxergam todas as lojas

Se um platform admin chamar endpoints de loja (GET /product/all, GET /order/all, GET /kitchen/orders...), o aspect **não ativa o filtro**. Não há storeId obrigatório nessas APIs. Na prática a web master **não monta** essas telas (ROLE_ROUTES só /admin/*), mas a API em si permite.

StoreAccess.assertOwned COM storeId == null dá return imediato — não bloqueia.

ALTO — User sem filtro Hibernate

Isolamento de funcionários depende de EmployeeService.requireStoreEmployee / findByStore_Id. Um endpoint genérico de User (não há hoje além de employees) seria perigoso. WaiterController lista waiters via service com filtro.

ALTO — Filhos sem store_id

OrderItem, OrderProductExtra, OrderMandatorySelection, MandatoryItem não têm tenant próprio. Segurança depende de só acessá-los via Order/MandatoryGroup já filtrados. Não há controller direto para OrderItem.

MÉDIO — Queries nativas

ProductRepository tem SQL nativo (DELETE FROM order_product...) que **ignora** filtro Hibernate.

MÉDIO — CategoryRepository.existsByName(String) sem store

Método existe; o service atual parece usar constraint (store_id, name). Risco latente se alguém passar a usar o método.

MÉDIO — Matching de comanda por nome

ComandaAccountService.matchesComanda também associa pedido de balcão se customerName contém "comanda" e o número. Pedidos de lojas diferentes ainda passam pelo filtro de order, mas o matching textual é frágil **dentro** da mesma loja.

BAIXO — Claim JWT storeId vs DB

Token antigo continua válido até exp; o store efetivo é o do banco. Se um usuário for movido de loja, o token não precisa ser reemitido. Se a intenção for "token amarra a loja", isso não acontece.

Conclusão sobre “Loja A acessa Loja B”

Para um STORE_ADMIN / WAITER / CASHIER / KITCHEN **com** storeId preenchido, o caminho feliz (filtro + existsByIdAndStore_Id + StoreAccess) impede listar/criar recursos da outra loja **nas entidades filtradas**.

O furo estrutural é o **platform admin sem store** e entidades filhas sem `store_id`. Um usuário de loja A **não** escolhe a loja B no cliente: o Android/Web nem enviam `storeId`. O isolamento de operadores de chão é razoável; o de platform admin contra dados de loja **não** está no SecurityConfig além de bloquear `/api/stores` ao contrário (só master acessa stores; master acessa o resto autenticado).

07 — Fluxo de pedido

Este é o fluxo principal de atendimento. O briefing descrevia Android → validações → cria pedido/itens → atualiza mesa/comanda → **atualiza estoque** → KDS/caixa.

O que o código realmente faz

- Criação: **somente** POST /tables/{tableId}/orders (GroupOrderController → GroupOrderService.createOrder → OrderService.addNewOrder).
- POST /order/new está @Deprecated e **lança** ORDER_ENDPOINT_DEPRECATED.
- A **web não cria pedidos**.
- Estoque **não** é decrementado.
- Preço: o backend **recalcula** subtotal a partir de Product + extras no servidor (calculateOpenSubtotal); o value enviado no DTO do Android não é a fonte da verdade no fechamento.

Diagrama ponta a ponta (classes reais)

```
flowchart TB
    subgraph android [Android]
        HF[HomeFragment / Tables / Comandas]
        CF[CategoriesFragment / ProductsFragment]
        Cart[CartViewModel.processOrder]
        UC[CreateOrderInGroupUseCase]
        GR[GroupOrderRepositoryImpl]
        API[GroupOrderAPI]
    end

    subgraph backend [Backend]
        GOC[GroupOrderController]
        GOS[GroupOrderService.createOrder]
        OS[OrderService.addNewOrder]
        PR[ProductRepository]
        TR[TableRepository]
        CR[ComandaRepository]
        WR[WaiterRepository]
        OR[OrderRepository]
        OIR[OrderItemRepository]
    end

    subgraph db [PostgreSQL]
        TBO[tb_order]
        TBI[tb_order_item]
        TBG[tb_group_order]
        TBT[tb_table / tb_comanda]
    end

    HF --> CF --> Cart --> UC --> GR --> API
    API -->|"POST /tables/{tableId}/orders"| GOC --> GOS
    GOS -->|"tableId != 999"| TBG
    GOS --> OS
    OS --> PR
    OS --> TR
    OS --> CR
    OS --> WR
    OS --> OR --> TBO
    OS --> OIR --> TBI
    GOS -->|"mesa real: isAvailable=false"| TBT
```

Três origens, um endpoint

Origem	tableId na URL	Body relevante	GroupOrder?
Mesa	id real da Table	waiterId, products, customerName	Sim: reusa ou cria GroupOrder ACTIVE
Comanda	999	comandaId, waiterId, products	Não. Order.comanda preenchido
Balcão	999	paymentMethod obrigatório, waiterId frequentemente 999	Não. Pedido standalone

Decisão em GroupOrderService.createOrder: tableId == OrderService.DEFAULT_ID (999) → não cria GroupOrder, chama orderService.addNewOrder(orderDTO) direto.

Validações em OrderService.addNewOrder

1. `customerName` obrigatório
2. `products` não vazio
3. `TenantContext.requireStoreId()`
4. Cada produto: `productRepository.existsByIdAndStore_Id`
5. `OrderMandatoryValidationService.validateMandatorySelections`
6. Categoria, se enviada: `existsByIdAndStore_Id`
7. `tableId` e `waiterId` obrigatórios no DTO
8. Se venda de balcão (`isCounterSale`: sem `comandaId` e `tableId` nulo ou 999): `paymentMethod` obrigatório
9. Se mesa real (`tableId != 999 && waiterId != 999`): mesa e waiter da loja
10. Se `comandaId`: comanda da loja; marca `Comanda.isAvailable=false`

Persistência:

- Order: store, kitchenStatus=NEW, status=ACTIVE, isFromTable, produtos ManyToMany order_product
- persistOrderItems → tb_order_item (snapshot productName, unitPrice, quantity)
- OrderProductExtra, OrderMandatorySelection
- Balcão: já grava `totalAmount` e `paymentMethod` na criação (o pedido **continua ACTIVE** até close ou até KDS DELIVERED em standalone)

Depois da criação, quem vê o pedido

Superfície	Como
KDS web	GET /kitchen/orders (polling 12s) — ACTIVE + FINALIZED 24h
Caixa web mesa	GET /tables/{id}/orders
Caixa web comanda	GET /comandas/{id}/orders
Android turno	GET /home/shift (polling 12s)
Android lista	GET /order/all
Dashboard loja	GET /dashboard/overview

O que o briefing previa e não acontece na criação

Esperado	Real
Atualiza estoque	Não
Fecha atendimento	Não (exceto total já calculado no balcão)
Push para KDS	Não; KDS puxa via HTTP
Web monta pedido	Não

Fluxos de mesa, comanda e balcão em separado: [08](#), [09](#), [10](#).

08 — Fluxo de mesa

Modelo

- Entidade Table → tb_table (number, isAvailable, capacity, isCounter, deleted, store_id)
- Conta da mesa: entidade GroupOrder → tb_group_order (status ACTIVE/CLOSED, totalAmount, paymentMethod, applyServiceTax, discount, table_id, store_id)
- Vários Order apontam para o mesmo GroupOrder (group_order_id)

Não existe “selecionar mesa no login”. A mesa é escolhida **depois**, no salão.

Cadastro de mesas

```
WEB Atendimento / Tables
→ tablesService
→ GET /tables/all          TableController / TableService
→ POST /tables/new         roles admin
→ DELETE /tables/delete?tableId=
```

Android só **lista**: TablesAPI GET tables/all. Não cria mesa.

Abrir mesa e lançar pedido

```
Android TablesFragment
→ GET /tables/all
→ usuário escolhe mesa
→ CartViewModel.setTableInfo(tableId, waiterId)
→ catálogo → CartViewModel.processOrder
→ POST /tables/{tableId}/orders
→ GroupOrderService.createOrder
  se não existe GroupOrder ACTIVE da mesa: cria
  OrderService.addNewOrder(dto, groupOrder)
  table.isAvailable = false
```

Pedidos seguintes na **mesma mesa** entram no mesmo GroupOrder ACTIVE (findActiveByTableId).

Consultar conta

Cliente	Endpoint
Web caixa	GET /tables/{tableId}/orders → GroupOrderDTO
Android	mesmo, via TablesAPI / GroupOrderAPI

Se tableId=999, o backend **não** trata como mesa: devolve pedidos de balcão (findAllCounterOrders). Ver [10-counter-flow.md](#).

Fechar mesa

```
WEB CheckoutDialog ou Android PaymentMethodDialog
→ POST /tables/{tableId}/group-orders/close
→ body CloseGroupOrderRequest { paymentMethod, applyServiceTax, discount }
→ GroupOrderService.closeActiveGroupOrder
```

Efeitos:

- Recusa se tableId==999 (COUNTER_GROUP_ORDER_CLOSE_NOT_ALLOWED)
- Exige paymentMethod
- Recusa se já CLOSED
- Calcula total (soma subtotais dos orders; × 1.10 se taxa; menos desconto)
- GroupOrder.status=CLOSED, closedAt, paymentMethod, totalAmount
- Cada order ainda ACTIVE → CLOSED, mesmo paymentMethod/discount/applyServiceTax
- table.isAvailable = true

Não altera kitchenStatus. **Não** mexe em Product.quantity.

Diagrama

```
flowchart LR
    subgraph android [Android]
        T[TablesFragment]
        C[CartViewModel]
    end
    subgraph api [Backend]
        GS[GroupOrderService]
    end
    subgraph db [DB]
        Table[tb_table]
        GO[tb_group_order]
        O[tb_order]
    end
    subgraph web [Web caixa]
        A[Atendimento]
        X[CheckoutDialog]
    end

    T -->|GET /tables/all| Table
    C -->|POST /tables/id/orders| GS
    GS --> GO
    GS --> O
    GS -->|isAvailable false| Table
    A -->|GET /tables/id/orders| GS
    X -->|POST .../group-orders/close| GS
    GS -->|CLOSED + isAvailable true| Table
```

Status da mesa vs status do pedido

Campo	Valores	Quando
Table.isAvailable	true/false	false no primeiro pedido da mesa; true no close do GroupOrder
GroupOrder.status	ACTIVE, CLOSED	um ACTIVE por mesa
Order.status	ACTIVE, CLOSED	CLOSED no fechamento da conta
Order.kitchenStatus	NEW ... FINALIZED	independente do close da mesa

09 — Fluxo de comanda

Modelo

- Entidade Comanda → `tb_comanda` (`number`, `isAvailable`, `deleted`, `store_id`)
- Não existe** `GroupOrder` de comanda. A “conta” é uma agregação em `ComandaAccountService` que devolve um `GroupOrderDTO` sintético (`groupOrderId=null`, `tableId=null`).
- Pedidos de comanda são `Order` com `comanda_id` preenchido, criados pela rota de **mesa 999**.

Cadastro

```
WEB Atendimento / Comandas
→ comandasService
→ GET /comandas/all
→ POST /comandas/new      admin
→ DELETE /comandas/delete?comandaId=
```

Android só lista: `ComandasAPI GET comandas/all`.

Lançar itens na comanda (Android)

```
ComandasFragment
→ CartViewModel.setComandaInfo(comandaId, comandaNumber)
   currentTableId = 999
→ processOrder
   finalTableId = 999
   body.comandaId = comandaId
   customerName default "Comanda {number}"
   paymentMethod = null
→ POST /tables/999/orders
→ GroupOrderService: ramo isCounter (999) → OrderService.addNewOrder
→ Order.comanda setado; Comanda.isAvailable = false
```

Isso **não** é venda de balcão (`isCounterSale` retorna `false` se `comandaId != null`), portanto **não** exige `paymentMethod` na criação.

Consultar conta

```
GET /comandas/{comandaId}/orders
→ ComandaOrderController
→ ComandaAccountService.getOpenAccount
→ findActiveByComandaId
→ também varre findActiveCounterOrders e matchesComanda
   (customerName contém "comanda" e o número)
```

Web: `accountService.getComandaAccount`. Android: `ComandasAPI.getGroupOrderByComanda`.

Fechar comanda

```
POST /comandas/{comandaId}/orders/close
body CloseCounterOrderRequest { paymentMethod, applyServiceTax, discount }
→ ComandaAccountService.closeOpenAccount
→ para cada order aberto: OrderService.closeCounterOrder
→ comanda.isAvailable = true
```

`closeCounterOrder` recusa pedido `isFromTable==true` (mesa). Comanda e balcão passam.

Diagrama

```
flowchart TB
    Android[Android[ComandasFragment + CartViewModel]]
```

```
API["POST /tables/999/orders<br/>comandaId no body"]
OS[OrderService.addNewOrder]
O[tb_order.comanda_id]
C[tb_comanda isAvailable=false]
GET["GET /comandas/{id}/orders"]
CAS[ComandaAccountService]
CLOSE["POST /comandas/{id}/orders/close"]

Android --> API --> OS --> O --> C
GET --> CAS --> O
CLOSE --> CAS
CAS -->|closeCounterOrder cada Order| O
CAS -->|isAvailable true| C
```

Diferença vs briefing

O briefing tratava comanda como irmã da mesa (tenant → commands → orders). No código:

- Comanda **não** gera `tb_group_order`
- Criação **reusa** o endpoint de mesa com sentinela 999
- Há matching extra por **texto** do `customerName` (`matchesComanda`) — frágil se dois nomes colidirem na mesma loja

10 — Fluxo de balcão

Balcão no código é o sentinela `tableId = 999` (`OrderService.DEFAULT_ID`, Android `COUNTER_TABLE_ID`). Não é uma linha real em `tb_table` obrigatória para o fluxo (o Android nem busca mesa 999).

Há também `Table.isCounter` na entidade, usado no cadastro de mesas, **não** no caminho `createOrder` de 999.

Início no Android

```
HomeFragment card BALCÃO
→ cartViewModel.startCounterSale()
  tableId = 999
  waiterId = 999
  comandaId = null
→ CategoriesFragment → ... → Cart
→ processOrder exige paymentType
→ POST /tables/999/orders
  paymentMethod no body
  fromTable = false
```

`GroupOrderService.createOrder` entra no ramo `isCounter` e **não** cria `GroupOrder`.

`OrderService.isCounterSale`: `comandaId == null` e (`tableId == null` OU `== 999`). Exige `paymentMethod`. Grava `totalAmount` já na criação; status permanece **ACTIVE**.

Fechar pedido de balcão

Dois caminhos:

A — Endpoint dedicado (Android `CloseCounterOrderUseCase`)

```
POST /counter/orders/{orderId}/close
→ CounterOrderController
→ OrderService.closeCounterOrder
→ status CLOSED, paymentMethod, total, closedAt
```

B — KDS marca DELIVERED em pedido standalone

`KitchenService.updateKitchenStatus`: SE `next == DELIVERED` e `OrderService.isStandaloneCounter(order) (!isFromTable && comanda == null)`, fecha o `Order` automaticamente (`status=CLOSED`).

A web de atendimento (`accountService`) **não** chama `/counter/orders/{id}/close`. O caixa web opera mesas e comandas. Pedido de balcão aparece no KDS e em `GET /order/all` / `GET /tables/999/orders` (lista de counter orders).

GET /tables/999/orders

`GroupOrderService.getActiveGroupOrder(999)` monta um `GroupOrderDTO` sintético COM `findAllCounterOrders()`, `groupOrderId=null`, `tableId=999`.

Fechar via `POST /tables/999/group-orders/close` é **recusado** (`COUNTER_GROUP_ORDER_CLOSE_NOT_ALLOWED`).

Diagrama

```
flowchart TB
  Home[HomeFragment.startCounterSale]
  Cart[CartViewModel paymentType obrigatório]
  POST["POST /tables/999/orders"]
  OS[OrderService.addNewOrder]
  O["tb_order ACTIVE + paymentMethod + totalAmount"]
  KDS["PATCH kitchen ... DELIVERED"]
  CLOSE["POST /counter/orders/{id}/close"]

  Home --> Cart --> POST --> OS --> O
  O --> KDS
  KDS -->|standalone| O
  O --> CLOSE
  CLOSE --> O
```

Comparação mesa / comanda / balcão

	Mesa	Comanda	Balcão
URL create	/tables/{idReal}/orders	/tables/999/orders	/tables/999/orders
GroupOrder	sim	não	não
Pagamento na criação	não	não	sim
Close	/tables/{id}/group-orders/close	/comandas/{id}/orders/close	/counter/orders/{id}/close ou KDS DELIVERED
Libera recurso	Table.isAvailable=true	Comanda.isAvailable=true	nenhum recurso de salão

11 — Fluxo da cozinha / KDS

O KDS **não é um app separado**. É a rota web `/app/cozinha` (`Kitchen.js`), permitida a `KITCHEN`, `ADMIN` e `STORE_ADMIN`.

Como o KDS obtém pedidos

Polling HTTP a cada 12 segundos + refresh manual. Sem WebSocket, SSE ou push.

```
Kitchen.js
setInterval(loadOrders, 12000)
→ kitchenService.getKitchenOrders()
→ GET /kitchen/orders
→ KitchenController
→ KitchenService.listActiveKitchenOrders
  OrderRepository.findActiveOrdersForKitchen
  + findFinalizedKitchenOrders(now - 24h)
```

Security: `GET /kitchen/**` exige role `KITCHEN` ou `admins`. **WAITER não lista o KDS**, embora possa `PATCH` status.

Status reais (`KitchenStatus`)

NEW → IN_PREPARATION → READY → DELIVERED → FINALIZED

Enum backend: `NEW`, `IN_PREPARATION`, `READY`, `DELIVERED`, `FINALIZED`.

Mapeamento na UI web (`Kitchen.js`):

UI	Enum
novo	NEW
em_producao	IN_PREPARATION
feito	READY
entregue	DELIVERED
finalizado (coluna opcional; FINALIZED também cai em entregue no mapa inverso)	FINALIZED / DELIVERED

Mudança:

```
PATCH /kitchen/orders/{orderId}/status
body KitchenStatusRequest { kitchenStatus }
→ KitchenService.updateKitchenStatus
→ StoreAccess.owned(order)
→ recusa se Order.status == CLOSED
→ setKitchenStatus
→ se READY | DELIVERED | FINALIZED e kitchenReadyAt null → agora
→ se DELIVERED e isStandaloneCounter → Order.status = CLOSED
```

Não há máquina de estados rígida: o service aceita qualquer valor do enum (`valueOf`), sem impedir pular etapas ou voltar.

Android e cozinha

Não há tela KDS. O garçom:

- Vê pedidos prontos via `GET /home/shift` (polling 12s em `MainActivity` → `ShiftAttentionStore` alerta sonoro/vibração)
- Pode `UpdateKitchenStatusUseCase` → `KitchenAPI` `PATCH /kitchen/orders/{id}/status` a partir de `GroupOrderViewModel`

Diagrama

```
flowchart LR
  Android[Android POST /tables/.../orders]
  OS[OS Order kitchenStatus=NEW status=ACTIVE]
  Poll[Kitchen.js polling 12s]
  GET[GET /kitchen/orders]
```

```
PATCH[PATCH /kitchen/orders/id/status]
Cols[novo → em_producao → feito → entregue]
```

```
Android --> OS
Poll --> GET --> OS
Cols --> PATCH --> OS
```

Relação com fechamento de conta

Fechar mesa/comanda **não** muda kitchenStatus. Um pedido pode estar order.status=CLOSED (pago) e ainda kitchenStatus=IN_PREPARATION, ou o inverso (pronto na cozinha, conta aberta).

Exceção: balcão standalone em DELIVERED fecha o order automaticamente.

Pedido já CLOSED **não** aceita mais PATCH de cozinha (COUNTER_ORDER_ALREADY_CLOSED).

12 — Fluxo de pagamento / caixa

Não existe entidade `Payment`, tabela `tb_payment` nem integração com PSP. Pagamento = enum `PaymentMethodEnum` gravado em `Order` e/ou `GroupOrder` no **fechamento**.

Valores: PIX, DEBIT, CREDIT, CASH, VOUCHER, OTHER.

Quem cobra

Papel	Superfície	O que fecha
CASHIER / STORE_ADMIN	Web /app/atendimento + CheckoutDialog	Mesa e comanda
WAITER / CASHIER	Android <code>PaymentMethodDialog</code> + <code>GroupOrderViewModel</code>	Mesa, comanda, balcão

A web **não** fecha pedido de balcão (`/counter/orders/{id}/close`).

Caixa web — passo a passo

```
Atendimento
→ lista mesas (tablesService GET /tables/all)
→ lista comandas (comandasService GET /comandas/all)
→ ao abrir conta:
  GET /tables/{id}/orders ou GET /comandas/{id}/orders
→ CheckoutDialog
  calcula subtotal na UI
  taxa 10% se applyServiceTax (subtotal * 0.1 no frontend)
  discount
  paymentMethod
→ POST /tables/{id}/group-orders/close
  ou POST /comandas/{id}/orders/close
```

O backend **recalcula** o total ($\times 1.10$ se taxa, menos desconto, floor em 0, scale 2). A taxa no frontend é espelho visual, não a fonte da persistência.

Android

Mesmos endpoints de close, body `PaymentRequest`. Balcão: pagamento já na **criação** (`paymentMethod` no `OrderCreateRequest`) e close posterior via `POST /counter/orders/{id}/close`.

Marks locais “pago” (`WaiterOrderMarks`) **não** persistem pagamento no servidor.

O que o fechamento grava

Mesa (`GroupOrderService.closeActiveGroupOrder`):

- `tb_group_order`: status CLOSED, `paymentMethod`, `totalAmount`, `applyServiceTax`, `discount`, `closedAt`
- cada `tb_order` do grupo: CLOSED, mesmos campos de pagamento, `totalAmount` = subtotal aberto do pedido (sem rateio da taxa no item — taxa está no group)
- `tb_table.isAvailable` = true

Comanda / balcão (`OrderService.closeCounterOrder`):

- um `tb_order`: CLOSED, `paymentMethod`, total com taxa/desconto aplicados naquele pedido
- comanda: `isAvailable` = true (no `ComandaAccountService`, depois do loop)

Estoque e dashboard

Fechar conta **não** baixa estoque. O dashboard (`GET /dashboard/overview`) lê pedidos/fechamentos para faturamento e operação.

Diagrama

```
flowchart TB
    Caixa[Caixa[Atendimento / CheckoutDialog]]
    GET[GET[GET conta mesa ou comanda]]
    UI[UI[UI: subtotal + 10% + desconto]]
    POST[POST[POST .../close]]
    Caixa --> GET
    GET --> UI
    UI --> POST
```

```
BE[GroupOrderService ou ComandaAccountService]
DB[(tb_group_order / tb_order)]
Mesa[tb_table ou tb_comanda isAvailable=true]

Caixa --> GET --> UI --> POST --> BE --> DB
BE --> Mesa
```

Gaps vs briefing

Esperado	Real
Pagamento salvo como registro próprio	Enum no pedido/grupo
Estoque no pagamento	Não
Gateway PIX/cartão	Não
Split / múltiplos meios na mesma conta	Um paymentMethod por close
Estorno	Não há endpoint

13 — Fluxo de estoque

Veredito

Não há movimento de estoque na venda. O briefing (Produto → ItemPedido → quantidade vendida → baixa → inventário) **não está implementado** como fluxo.

O que existe:

- Campo `Product.quantity` (Long) em `tb_product`
- CRUD via `ProductService` / `POST|PUT /product/*` (web catálogo)
- Dashboard trata quantidade **> 0 e ≤ 8** como estoque crítico (`DashboardService.LOW_STOCK_MAX = 8`, alerta `LOW_STOCK`)
- Tela web `Inventory.js` é **mock** e **não está montada** no Dashboard

Quando `quantity` muda

Somente no cadastro/edição de produto (`ProductService` copia `productDTO.getQuantity()`).

Não há `setQuantity` decrementando em `OrderService`, `GroupOrderService`, `ComandaAccountService` OU `KitchenService`.

`OrderItem.quantity` é a **quantidade vendida naquele item**, snapshot; não atualiza `Product.quantity`.

O que não existe

- Baixa na criação do pedido
- Baixa na confirmação/KDS
- Baixa no pagamento/fechamento
- Cancelamento com estorno de estoque
- Devolução
- Ajuste de inventário com histórico
- Reposição como entidade
- Endpoint de inventário
- Android lendo/escrevendo estoque (o `Product.quantity` pode vir no JSON do catálogo, mas não há tela de estoque)

Diagrama do que o código faz

```

flowchart LR
    Cadastro[Web Menu ProductFormDialog]
    API[PUT/POST /product]
    P[tb_product.quantity]
    Dash[GET /dashboard/overview]
    Alert[criticalStock se 1..8]
    Venda[POST /tables/.../orders]
    Item[tb_order_item.quantity]

    Cadastro --> API --> P --> Dash --> Alert
    Venda --> Item
    Venda -. -> |não altera| P
  
```

Status na matriz: **parcial** no backend (campo + alerta), **não implementado** como módulo de inventário, **não montado** na web, **ausente** no Android operacional.

14 — Mapa de endpoints

Base web: REACT_APP_API_BASE_URL (default `http://localhost:8080`).

Base Android: `BuildConfig.BASE_URL`.

Autenticação padrão: `Authorization: Bearer <JWT>`, exceto login e assets públicos.

Legenda origem: **Web**, **Android**, **Nenhum cliente** (só API), **Deprecated**.

Auth e plataforma

Método	Endpoint	Origem	Controller	Service	Repository	Entidade
POST	/api/auth/login	Web, Android	AuthController	AuthService	UserRepository	User
GET	/api/auth/me	Nenhum cliente	AuthController	AuthService	UserRepository	User
GET	/api/stores	Web master	StoreController	StoreService	StoreRepository	Store
GET	/api/stores/{id}	Service web definido, não chamado	StoreController	StoreService	StoreRepository	Store
GET	/api/stores/dashboard	Web master	StoreController	StoreService	StoreRepository	Store
POST	/api/stores	Web master	StoreController	StoreService	StoreRepository, UserRepository	Store, User, Subscription
PUT	/api/stores/{id}	Web master	StoreController	StoreService	idem	Store, User, Subscription

Colaboradores e garçons

Método	Endpoint	Origem	Controller	Service	Repository	Entidade
GET	/api/employees	Web	EmployeeController	EmployeeService	UserRepository	User
POST	/api/employees	Web	EmployeeController	EmployeeService	UserRepository, WaiterRepository	User, Waiter
PUT	/api/employees/{id}	Web	EmployeeController	EmployeeService	idem	User
DELETE	/api/employees/{id}	Web	EmployeeController	EmployeeService	idem	User
GET	/api/waiters	Nenhum cliente mapeado	WaiterController	WaiterService	WaiterRepository	Waiter
GET	/api/waiters/active	Nenhum cliente	WaiterController	WaiterService	WaiterRepository	Waiter
GET	/api/waiters/{id}	Nenhum cliente	WaiterController	WaiterService	WaiterRepository	Waiter
POST	/api/waiters	Nenhum cliente	WaiterController	WaiterService	WaiterRepository	Waiter
PUT	/api/waiters/{id}	Nenhum cliente	WaiterController	WaiterService	WaiterRepository	Waiter
DELETE	/api/waiters/{id}	Nenhum cliente	WaiterController	WaiterService	WaiterRepository	Waiter

Catálogo

Método	Endpoint	Origem	Controller	Service	Repository	Entidade
GET	/category/all	Web, Android	CategoryController	CategoryService	CategoryRepository	Category
GET	/category/getById	Nenhum cliente	CategoryController	CategoryService	CategoryRepository	Category
POST	/category/new	Web	CategoryController	CategoryService	CategoryRepository	Category
PUT	/category/update?categoryId=	Web	CategoryController	CategoryService	CategoryRepository	Category
DELETE	/category/deleteById?categoryId=	Web	CategoryController	CategoryService	CategoryRepository	Category
DELETE	/category/delete/all	Nenhum cliente	CategoryController	CategoryService	CategoryRepository	Category
GET	/product/all	Web, Android	ProductController	ProductService	ProductRepository	Product
GET	/product/getByCategoryId?categoryId=	Web, Android	ProductController	ProductService	ProductRepository	Product
GET	/product/getByProductId?productId=	Nenhum cliente	ProductController	ProductService	ProductRepository	Product

POST	/product/new	Web (Android API existe)	ProductController	ProductService	ProductRepository	Product
PUT	/product/update?productId=	Web	ProductController	ProductService	ProductRepository	Product
DELETE	/product/delete?productId=	Web	ProductController	ProductService	ProductRepository	Product
DELETE	/product/delete/all	Nenhum cliente	ProductController	ProductService	ProductRepository	Product
POST	/product/extra/new	Android GET extras; POST não mapeado na web	ProductExtraController	ProductExtraService	ProductExtraRepository	ProductExtra
GET	/product/extra	Android	ProductExtraController	ProductExtraService	ProductExtraRepository	ProductExtra
GET	/product/extra/{id}	Android	ProductExtraController	ProductExtraService	ProductExtraRepository	ProductExtra
GET	/product/extra/byProduct?productId=	Android	ProductExtraController	ProductExtraService	ProductExtraRepository	ProductExtra
PUT	/product/extra/{id}	Nenhum cliente web	ProductExtraController	ProductExtraService	ProductExtraRepository	ProductExtra
DELETE	/product/extra/{id}	Nenhum cliente web	ProductExtraController	ProductExtraService	ProductExtraRepository	ProductExtra
POST	/product/mandatory-groups	Nenhum cliente web mapeado	MandatoryGroupController	MandatoryGroupService	MandatoryGroupRepository	MandatoryGroup
POST	/product/mandatory-groups/products/{productId}/assign/{groupId}	Nenhum cliente web mapeado	MandatoryGroupController	MandatoryGroupService	—	Product/MandatoryGroup
GET	/product/mandatory-groups/products/{productId}	Nenhum cliente web mapeado (Android lê via product DTO)	MandatoryGroupController	MandatoryGroupService	—	MandatoryGroup

Mesas, comandas, pedidos

Método	Endpoint	Origem	Controller	Service	Repository	Entidade
GET	/tables/all	Web, Android	TableController	TableService	TableRepository	Table
GET	/tables/available	Nenhum cliente	TableController	TableService	TableRepository	Table
GET	/tables/unavailable	Nenhum cliente	TableController	TableService	TableRepository	Table
POST	/tables/new	Web	TableController	TableService	TableRepository	Table
DELETE	/tables/delete?tableId=	Web	TableController	TableService	TableRepository	Table
DELETE	/tables/delete/all	Nenhum cliente	TableController	TableService	TableRepository	Table
POST	/tables/{tableId}/orders	Android	GroupOrderController	GroupOrderService + OrderService	GroupOrderRepository, OrderRepository	GroupOrder, Order, OrderItem
GET	/tables/{tableId}/orders	Web, Android	GroupOrderController	GroupOrderService	idem	GroupOrder / sintético 999
POST	/tables/{tableId}/group-orders/close	Web, Android	GroupOrderController	GroupOrderService	idem	GroupOrder, Order, Table
GET	/comandas/all	Web, Android	ComandaController	ComandaService	ComandaRepository	Comanda

POST	/comandas/new	Web	ComandaController	ComandaService	ComandaRepository	Comanda
DELETE	/comandas/delete?comandaId=	Web	ComandaController	ComandaService	ComandaRepository	Comanda
GET	/comandas/{comandaId}/orders	Web, Android	ComandaOrderController	ComandaAccountService	OrderRepository	Order (DTO GroupOrder)
POST	/comandas/{comandaId}/orders/close	Web, Android	ComandaOrderController	ComandaAccountService	OrderRepository	Order, Comanda
POST	/counter/orders/{orderId}/close	Android	CounterOrderController	OrderService	OrderRepository	Order
GET	/order/all	Android	OrderController	OrderService	OrderRepository	Order
POST	/order/new	Deprecated (Android ainda declara)	OrderController	—	—	—
GET	/order/getByTable	Deprecated	OrderController	—	—	—
PUT	/order/edit	Deprecated	OrderController	—	—	—

Cozinha, home, dashboard

Método	Endpoint	Origem	Controller	Service	Repository	Entidade
GET	/kitchen/orders	Web KDS	KitchenController	KitchenService	OrderRepository	Order
PATCH	/kitchen/orders/{orderId}/status	Web, Android	KitchenController	KitchenService	OrderRepository	Order
GET	/dashboard/overview?period=	Web	DashboardController	DashboardService	vários	agregados
GET	/home/options/all	Android	HomeController	estático	—	HomeDTO
GET	/home/shift	Android	ShiftOverviewController	ShiftOverviewService	OrderRepository	Order

Público / docs

Método	Endpoint	Notas
GET	/swagger-ui/**, /v3/api-docs/**	público
GET	/images/**	público
OPTIONS	/**	CORS

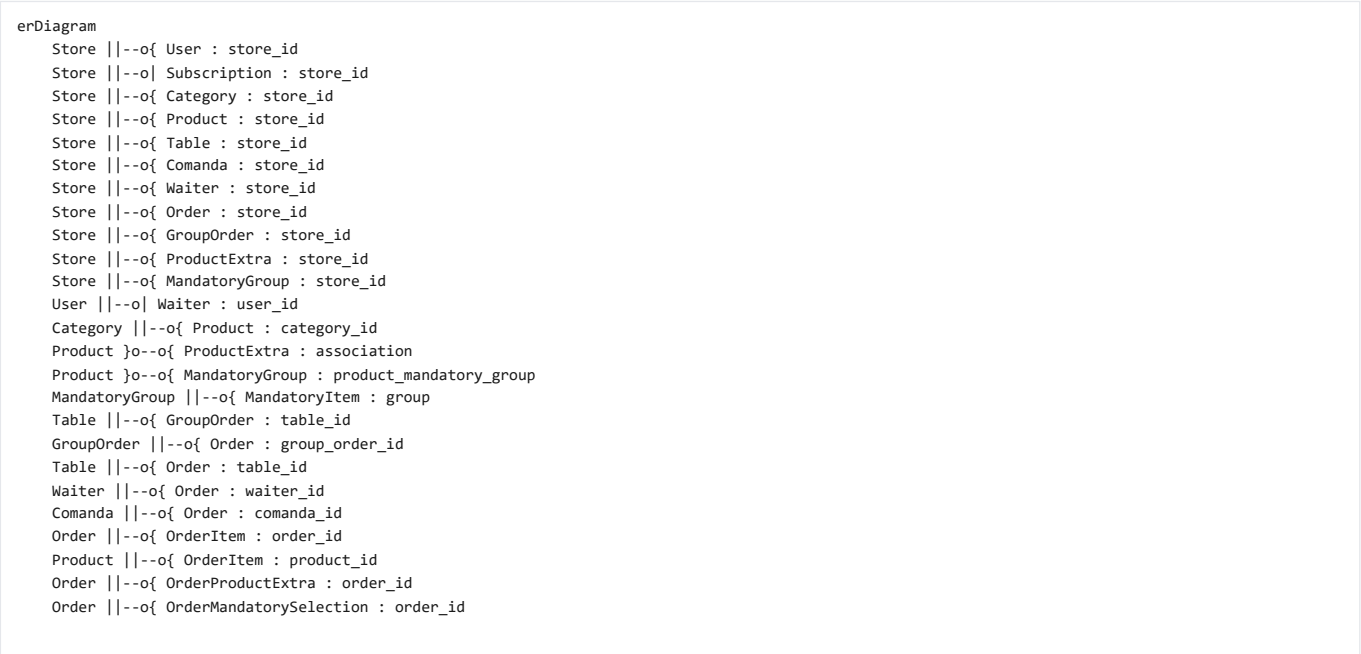
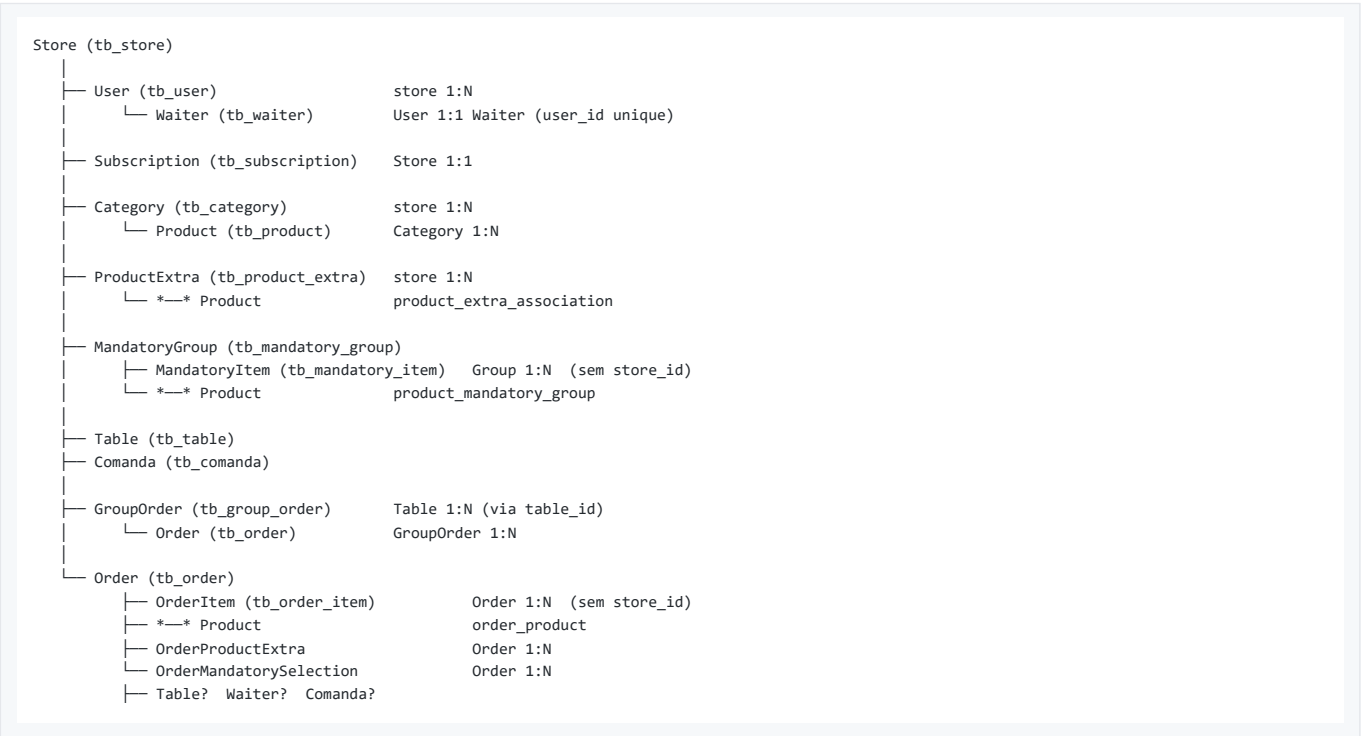
Prefixos mistos são reais: recursos novos em /api/, legado em /category, /product, /tables, /order, /kitchen.

15 — Modelo de dados

Schema gerenciado por Hibernate ddl-auto=update. Sem Flyway/Liquibase. Banco PostgreSQL, database orderix.

Campo de tenant: store_id (exceto tabelas filhas listadas).

Diagrama de entidades (nomes reais)



Cardinalidade

Relação	Tipo
Store 1:N User	User.store
Store 1:1 Subscription	Subscription.store unique

Store 1:N Category, Product, Table, Comanda, Waiter, Order, GroupOrder, ProductExtra, MandatoryGroup	store_id
Category 1:N Product	Product.category
User 1:1 Waiter	Waiter.user unique
Table 1:N GroupOrder	GroupOrder.table obrigatório
GroupOrder 1:N Order	Order.groupOrder opcional
Table 1:N Order	Order.table opcional
Waiter 1:N Order	Order.waiter opcional
Comanda 1:N Order	Order.comanda opcional
Order 1:N OrderItem	OrderItem.order
Product 1:N OrderItem	OrderItem.productId (Long, não @ManyToOne)
Order N:N Product	join order_product
Product N:N ProductExtra	product_extra_association
Product N:N MandatoryGroup	product_mandatory_group
MandatoryGroup 1:N MandatoryItem	

Tabelas e campos principais

Tabela	Entidade	Campos relevantes
tb_store	Store	name, slug unique, responsible_name, email, phone, active, createdAt, updatedAt
tb_user	User	store_id, name, username unique, email unique, password_hash, role, active
tb_subscription	Subscription	store_id unique, plan, price, status, startDate, expiresAt
tb_category	Category	store_id, name, unique (store_id, name), background_color, image TEXT, active, parent_id
tb_product	Product	store_id, name, image TEXT, portion, observation, isAvailable, value, quantity, category_id
tb_product_extra	ProductExtra	store_id + extra
tb_mandatory_group / tb_mandatory_item	grupos obrigatórios	item sem store_id
tb_table	Table	store_id, number, isAvailable, capacity, isCounter, deleted
tb_comanda	Comanda	store_id, number, isAvailable, deleted
tb_waiter	Waiter	store_id, user_id, name, email, isActive, isCounter
tb_group_order	GroupOrder	store_id, table_id, status, totalAmount, paymentMethod, applyServiceTax, discount, createdAt, closedAt
tb_order	Order	store_id, table_id, group_order_id, waiter_id, comanda_id, customerName, observation, isFromTable, status, totalAmount, paymentMethod, closedAt, applyServiceTax, discount, kitchenStatus, kitchen_ready_at
tb_order_item	OrderItem	order_id, product_id, product_name, unit_price, quantity, observation
tb_order_product_extra	OrderProductExtra	order, product, extra, quantity
tb_order_mandatory_selection	OrderMandatorySelection	order, group, item

Enums persistidos (STRING)

Enum	Valores
Role	MASTER, SUPER_ADMIN, ADMIN, STORE_ADMIN, WAITER, KITCHEN, CASHIER
OrderStatus	ACTIVE, CLOSED
KitchenStatus	NEW, IN_PREPARATION, READY, DELIVERED, FINALIZED
GroupOrderStatus	ACTIVE, CLOSED
PaymentMethodEnum	PIX, DEBIT, CREDIT, CASH, VOUCHER, OTHER
SubscriptionPlan	BASIC, PRO, PREMIUM
SubscriptionStatus	ACTIVE, PENDING, OVERDUE, BLOCKED, CANCELED

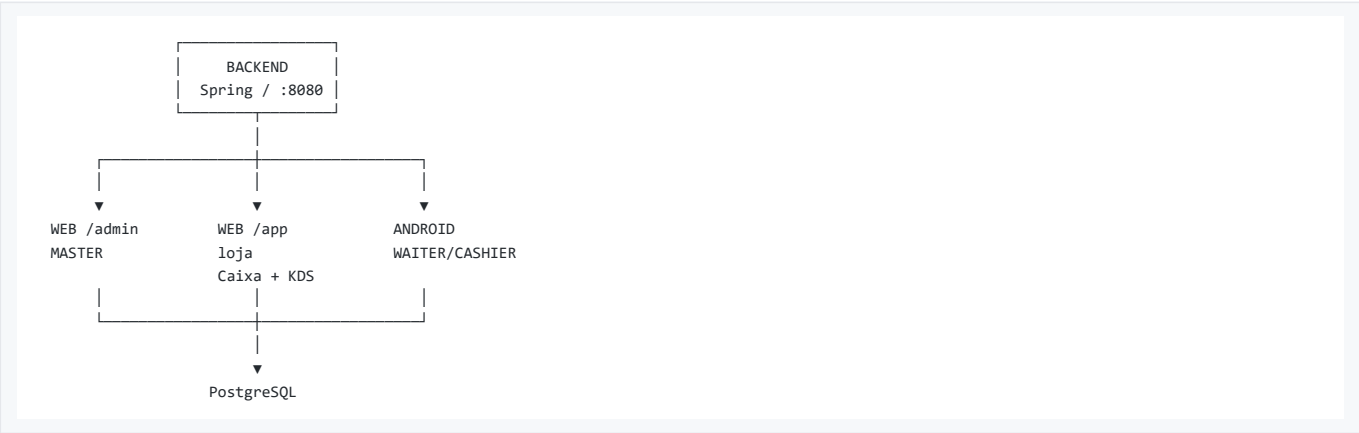
O que não existe no modelo

- Tabela de pagamentos

- Tabela de movimento de estoque / inventário
- Tabela de turnos (shift é DTO agregado em memória)
- tenantId genérico além de store_id
- Soft delete uniforme (só Table/Comanda têm deleted)

16 — Mapa de integrações e dependências

Topologia



```
flowchart TB
    subgraph web [orderix-web]
        Admin["/admin MASTER"]
        App["/app loja"]
        KDS["/app/cozinha"]
        Caixa["/app/atendimento"]
    end
    subgraph android [orderix-android]
        Garcom["Garçom / Caixa"]
        Print["ThermalPrinter Bluetooth"]
    end
    API[orderix-backend]
    PG[(PostgreSQL)]

    Admin --> API
    App --> API
    KDS --> API
    Caixa --> API
    Garcom --> API
    Garcom --> Print
    API --> PG
```

Web e Android **não** se integram. A impressora é só dispositivo local no Android. Firebase Crashlytics (Android) e Firebase init (web) não fazem parte do fluxo de pedidos.

Quem depende de quais módulos de API

Módulo backend	Web admin	Web loja	Web KDS	Web caixa	Android
/api/auth/login	✓	✓	✓	✓	✓
/api/stores	✓	✗	✗	✗	✗
/api/employees	✗	✓	✗	✗	✗
/api/waiters	✗	✗	✗	✗	✗
/category, /product CRUD	✗	✓	✗	✗	leitura + extras
/tables CRUD	✗	✓ (via atendimento)	✗	✓	leitura
/comandas CRUD	✗	✓	✗	✓	leitura
POST /tables/{id}/orders	✗	✗	✗	✗	✓ único criador
GET conta + POST close mesa/comanda	✗	✗	✗	✓	✓
POST /counter/orders/{id}/close	✗	✗	✗	✗	✓
GET /kitchen/orders	✗	✗	✓	✗	✗
PATCH kitchen status	✗	✗	✓	✗	✓
GET /dashboard/overview	✗	✓	✗	✗	✗

GET /home/*	✗	✗	✗	✗	✓
GET /order/all	✗	✗	✗	✗	✓

Integrações externas

Sistema	Presente?
Gateway de pagamento	Não
E-mail / SMTP	Não
SMS	Não
Object storage	Não (imagem TEXT/base64 no JSON)
Firebase Auth	Inicializado no web, não usado
Firebase Crashlytics	Android sim
FCM	Não
Impressora térmica	Android Bluetooth ESCPOS
OpenAPI/Swagger	Backend local

Variáveis de ambiente (nomes)

Backend: PORT, DB_URL, DB_USER, DB_PASS, DDL_OPTION, CORS_ALLOWED_ORIGINS, CORS_ALLOW_CREDENTIALS, JWT_SECRET, JWT_EXPIRATION_MS, BOOTSTRAP_SUPER_USER, BOOTSTRAP_SUPER_PASSWORD, DEMO_SEED.

Web: REACT_APP_API_BASE_URL, REACT_APP_FIREBASE_.*.

Android: debug.api.url / -PdebugApiUrl; release BASE_URL placeholder Render.

Acoplamento operacional

O estabelecimento só funciona completo se:

- 1. Master criou a store (web admin)
- 2. Admin cadastrou catálogo + mesas/comandas + colaboradores com role e senha (web loja)
- 3. Garçom/caixa com waiter vinculado faz login no Android e lança pedidos
- 4. Cozinha usa a **mesma API** pela web KDS
- 5. Caixa fecha na web ou no Android

Se o Android estiver fora: a loja **não lança pedidos de salão** (a web não chama POST /tables/{id}/orders).

Se a web KDS estiver fora: a cozinha não tem board; o garçom ainda vê GET /home/shift.

Se a web caixa estiver fora: o Android ainda fecha mesa/comanda/balcão.

17 — Problemas, gaps e inconsistências

Somente itens com correspondência no código. Classificação: **CRÍTICO** / **ALTO** / **MÉDIO** / **BAIXO**.

Critério de "implementado": UI → ação → client → endpoint → controller → service → repository → banco → response → UI. Elo ausente = parcial.

CRÍTICO

C1. Sem baixa de estoque na venda

`Product.quantity` não é decrementado em nenhum service de pedido. Dashboard alerta 1–8 unidades como se fosse estoque real. Risco operacional: números de estoque mentem após a primeira venda.

C2. Platform admin sem `storeId` desliga o filtro Hibernate

`StoreHibernateFilterAspect` + `StoreAccess` com `storeId == null` não isolam dados de loja. `GET /product/all, /order/all, /kitchen/orders` etc. ficam acessíveis a MASTER autenticado. A UI master não expõe essas telas, mas a API sim.

C3. Pedidos só nascem no Android

Nenhum fluxo web chama `POST /tables/{tableId}/orders`. Loja só com web **não consegue lançar pedido de atendimento**.

ALTO

A1. Sem realtime no KDS

Polling 12s. Cozinha e salão podem divergir por até esse intervalo. Sem lock otimista: dois clientes PATCH o mesmo pedido por último ganha.

A2. Fechar mesa não sincroniza cozinha (e vice-versa)

`Order.status` CLOSED e `kitchenStatus` são independentes, exceto balcão DELIVERED. Caixa pode cobrar pedido ainda NEW; cozinha não altera pedido já CLOSED.

A3. Endpoints `/order/new|edit|getByTable` deprecated mas Android ainda declara `POST order/new`

O carrinho usa `GroupOrderAPI` (ok). Se algum caminho residual chamar `OrderAPI.createOrderWithProducts`, o backend lança erro.

A4. Matching de comanda por `customerName`

`ComandaAccountService.matchesComanda` associa pedidos cujo nome contém "comanda" + número. Colisão possível na mesma loja.

A5. Sem refresh token

401 derruba sessão. JWT 12h. `RefreshTokenInterceptor` no Android não refresha.

A6. Entidades filhas sem `store_id`

`OrderItem`, `OrderProductExtra`, `OrderMandatorySelection`, `MandatoryItem`. Isolamento só via pai.

A7. Tela de estoque e configurações na web são mock e não montadas

Paths liberados no `accessControl`; `screenBySlug` não inclui. Usuário `STORE_ADMIN` pode ter rota permitida sem UI.

MÉDIO

M1. `WaiterController` sem cliente

Web usa `EmployeeController` (que chama `ensureForUser`). Android não chama `/api/waiters`. API de waiters parece órfã.

M2. `GET /api/auth/me` sem cliente

M3. Prefixos de API inconsistentes (`/api/*` vs legado)

Funciona, mas complica gateway/CORS/docs.

M4. Taxa de serviço 10% duplicada (web UI e backend)

Se um lado mudar, a prévia do caixa mente até o POST.

M5. `OrdersViewModel` Android força display (`isPaid=true`, `saleType="balcao"`, `orderStatus="fazendo"`)

Lista de pedidos pode mentir na UI.

M6. `CounterOrderFragment` sem navegação a partir do Home

Balcão vai a categorias; fragmento no graph parece legado.

M7. Forgot password e registro: stubs

Web e Android sem backend.

M8. Firebase Auth morto no web; Google login desabilitado**M9. Master: gráfico de receita parcialmente mock (`mockRevenueHistory.js`); `getStoreById` nunca chamado****M10. Queries nativas em `ProductRepository` ignoram filtro Hibernate****M11. `CategoryRepository.existsByName(String)` sem store — método latente****M12. Kitchen status sem máquina de estados**

Qualquer enum é aceito; dá para pular ou voltar.

M13. Badge de notificações hardcoded "4" em `Dashboard.js`**M14. Imagens de cardápio em data URL/base64 no JSON**

Payload grande; filesystem efêmero no Render de qualquer forma.

M15. Release Android `BASE_URL` placeholder Render

App de produção não aponta para host real sem build extra.

M16. Sem cancelamento / estorno / devolução de pedido**M17. Concorrência ao fechar mesa**

Dois caixas no mesmo `GroupOrder` ACTIVE: o segundo toma `GROUP_ORDER_ALREADY_CLOSED` ou condição de corrida conforme timing. Sem versão/lock explícito além do status.

M18. `GET /kitchen/**` bloqueia WAITER; PATCH permite

Garçom atualiza status sem ver o board KDS (intencional ou não, assim está).

BAIXO**B1. `OrderConfirmationViewModel` vazio****B2. Pastas Android vazias (`checkout`, `tableSales`, `contact`)****B3. README Android desatualizado (Room, Picasso, Firestore)****B4. Plugin Compose no Gradle root sem UI Compose****B5. `ROLE_PERMISSIONS` stub na web****B6. `CreateProductUseCase` no Android — catálogo é responsabilidade da web****B7. Dual modelo de produtos no pedido: ManyToMany `order_product` e `tb_order_item`**

Redundância; risco de divergência se um lado for atualizado e o outro não (`OrderService` grava os dois na criação).

B8. `WaiterHome` web placeholder

B9. Assinatura: planos na web (PLANS) espelham backend; telas MasterPlans/MasterSubscriptions não têm API própria de “planos”
— o plano vai no `CreateStoreRequest`

Matriz rápida de elos quebrados

Sintoma	Elo ausente
Inventário	UI mock + sem service de movimento + sem baixa no OrderService
Esqueci senha	UI sem endpoint
Seleção de estabelecimento no Android	Não há tela; User já tem Store
Pedido na web	Sem chamada POST create
Pagamento PSP	Sem integração
Sync offline Android	Sem Room
KDS push	Sem WebSocket
/api/waiters	Sem tela
POST /order/new	Controller lança deprecated

18 — Fluxo completo do sistema

Documento principal. Compara o fluxo **esperado** (briefing) com o **código real**. Responde às 20 perguntas de rastreabilidade.

1. Visão esperada vs realidade

ESPERADO	REAL NO CÓDIGO
MASTER ADMIN	MASTER/SUPER_ADMIN web /admin
cria estabelecimento	POST /api/stores → Store + STORE_ADMIN + Subscription
loja	Store (tb_store), tenant = storeId
cadastro funcionários	POST /api/employees (+ Waiter se WAITER/CASHIER)
cadastro categorias/produtos	/category/* /product/* (web)
cadastro mesas/comandas	/tables/* /comandas/* (web)
garçom Android	Sim, só WAITER/CASHIER com waiterId
seleciona estabelecimento	NÃO. storeId vem do User
Mesa / Comanda / Balcão	Sim (balcão e comanda usam tableId 999)
Pedido → Backend → Banco	POST /tables/{id}/orders
cozinha KDS app	NÃO. É rota web /app/cozinha + polling 12s
Caixa web fecha	Sim, mesa e comanda (não balcão)
estoque na venda	NÃO
Pagamento gateway	NÃO. Enum no close

flowchart TB

Master[MASTER /admin]

Store[StoreService.createStore]

Admin[STORE_ADMIN /app]

Cat[Categorias e produtos]

Floor[Mesas e comandas]

Emp[Colaboradores]

And[Android WAITER]

API[Backend]

PG[(PostgreSQL)]

KDS[Web Kitchen polling]

Caixa[Web Atendimento]

Master --> Store --> PG

Admin --> Cat --> API --> PG

Admin --> Floor --> API

Admin --> Emp --> API

And --> |POST /tables/id/orders| API

API --> KDS

API --> Caixa

Caixa --> |POST close| API

2. Jornada completa (código)

Fase A — Plataforma cria a loja

Usuário MASTER
→ LoginForm → POST /api/auth/login
→ /admin/restaurantes
→ storesService.createStore
→ POST /api/stores
→ StoreController → StoreService.createStore
tb_store
tb_user (STORE_ADMIN)
tb_subscription

Credenciais: adminEmail/adminUsername + adminPassword NO CreateStoreRequest. Role fixa STORE_ADMIN.

Fase B — Loja configura operação

STORE_ADMIN → /app
Colaboradores → CRUD /api/employees
role WAITER/CASHIER → WaiterService.ensureForUser → tb_waiter

```

    role KITCHEN → User sem Waiter (floor operator só WAITER/CASHIER)
    Catálogo → /category + /product
    Mesas → POST /tables/new
    Comandas → POST /comandas/new

```

Fase C — Garçom vende

```

Android LoginActivity → POST api/auth/login
  rejeita se não WAITER/CASHIER ou waiterId null
  DataStore: token + storeId + waiterId
MainActivity / HomeFragment
  GET home/options/all (cards estáticos Balcão, Mesas, Comandas)
  GET home/shift (polling 12s)

Mesa: TablesFragment → POST /tables/{id}/orders
Comanda: setComandaInfo → POST /tables/999/orders + comandaId
Balcão: startCounterSale → POST /tables/999/orders + paymentMethod

OrderService.addNewOrder
  valida store, produtos, mesa/waiter/comanda
  tb_order status=ACTIVE kitchenStatus=NEW
  tb_order_item + extras + mandatory
  mesa/comanda isAvailable=false
  NÃO mexe em Product.quantity

```

Fase D — Cozinha

```

KITCHEN → /app/cozinha
  GET /kitchen/orders a cada 12s
  PATCH /kitchen/orders/{id}/status
    NEW | IN_PREPARATION | READY | DELIVERED | FINALIZED
  DELIVERED em balcão standalone → Order CLOSED

```

Fase E — Caixa

```

CASHIER → /app/atendimento
  GET /tables/{id}/orders ou GET /comandas/{id}/orders
  CheckoutDialog → POST .../close { paymentMethod, applyServiceTax, discount }
  GroupOrder/Orders CLOSED, recurso liberado

Android também fecha (incluindo POST /counter/orders/{id}/close)

```

Fase F — Gestão

```

STORE_ADMIN → /app/dashboard
  GET /dashboard/overview?period=
    faturamento, operação, criticalStock (quantity 1..8)

```

3. Respostas rápidas

1. Como Web, Android e Backend conversam?

Só REST + JWT. Clientes → Spring → PostgreSQL. Sem canal entre Web e Android.

2. Quais endpoints existem?

Ver [14-api-map.md](#) (~60 incluindo deprecated).

3. Quem usa cada endpoint?

Pedido create = Android. KDS GET = web. Close mesa/comanda = web e Android. Stores = só web master. Waiters API = nenhum cliente.

4. Login?

POST /api/auth/login → BCrypt + loja ativa → JWT. Web localStorage; Android DataStore. Sem refresh. Detalhe em [05](#).

5. Multi-tenant?

storeId do User, store, TenantContext, Hibernate storeFilter. Não vai em header. Riscos em [06](#).

6. Como um pedido nasce?

Android CartViewModel.processOrder → POST /tables/{tableId}/orders → GroupOrderService + OrderService.addNewOrder. [07](#).

7. Como chega na cozinha?

Persistido com kitchenStatus=NEW. KDS puxa GET /kitchen/orders (poll 12s).

8. Cozinha altera status?

PATCH /kitchen/orders/{orderId}/status. Sem transições obrigatórias.

9. Caixa recupera o pedido?

GET /tables/{id}/orders OU GET /comandas/{id}/orders. Balcão: Android/GET /tables/999/orders / GET /order/all.

10. Mesa/comanda fechada?

POST close correspondente; isAvailable=true; orders CLOSED. [08](#) [09](#).

11. Quando o estoque é atualizado?

Só no CRUD de produto. **Nunca** na venda. [13](#).

12. Como o pagamento é registrado?

PaymentMethodEnum em Order/GroupOrder no close. Sem tabela Payment. [12](#).

13. Tabelas?

tb_store, tb_user, tb_subscription, tb_category, tb_product, tb_table, tb_comanda, tb_waiter, tb_order, tb_order_item, tb_group_order, extras/mandatory. [15](#).

14–17. **Completas / parciais / quebradas / inexistentes** — matriz abaixo.

18. Riscos de segurança/multi-tenancy?

Platform admin sem filtro; filhos sem store_id; matching textual de comanda; queries nativas. [06](#) [17](#).

19. Dependências?

Android depende do catálogo/mesas cadastrados na web. KDS e caixa dependem de pedidos do Android. [16](#).

20. Fluxo cadastro → venda?

Fases A–F neste documento.

4. Matriz de rastreabilidade

Critério: UI → client → endpoint → service → DB → UI.

Funcionalidade	Web	Android	Backend	Banco	Status
Login JWT	✓	✓	✓	✓ tb_user	Completo
Logout / 401 limpa sessão	✓	✓	✓ filtro	—	Completo
Refresh token	✗	✗ interceptor não refresca	✗	✗	Não implementado
Forgot password	⚠ UI stub	⚠ stub	✗	✗	Incompleto
Cadastro de loja	✓ /admin	✗	✓ StoreService	✓	Completo (só master web)
Assinatura / bloqueio 403	✓ redirect	⚠ (403 genérico)	✓ filtro	✓ tb_subscription	Parcial
Planos master (CRUD API)	⚠ telas + mock receita	✗	⚠ plano no create store	✓ enum	Parcial
Colaboradores	✓	✗	✓ EmployeeService	✓ tb_user	Completo no web
Waiter API dedicada	✗	✗	✓ WaiterController	✓ tb_waiter	Órfão no cliente
Categorias	✓ CRUD	✓ leitura	✓	✓	Completo
Produtos	✓ CRUD	✓ leitura	✓	✓	Completo
Extras / mandatory	⚠ cadastro web pouco mapeado nos services HTTP	✓ leitura no detalhe	✓	✓	Parcial
Mesas cadastro	✓	✗ leitura	✓	✓	Completo (cadastro web)
Comandas cadastro	✓	✗ leitura	✓	✓	Completo (cadastro web)
Seleção de loja no app	✗	✗	User já tem store	✓	Não existe (por desenho)

Pedido mesa	✗ não cria	✓	✓	✓	Completo só Android→API
Pedido comanda	✗ não cria	✓ via table 999	✓	✓	Completo só Android→API
Pedido balcão	✗ não cria	✓	✓	✓	Completo só Android→API
Listar conta mesa	✓	✓	✓	✓	Completo
Listar conta comanda	✓	✓	✓	✓	Completo
Fechar mesa	✓	✓	✓	✓	Completo
Fechar comanda	✓	✓	✓	✓	Completo
Fechar balcão	✗	✓	✓	✓	Parcial (só Android)
KDS board	✓ poll 12s	✗	✓	✓	Completo na web
Status cozinha PATCH	✓	✓	✓	✓	Completo
WebSocket/SSE	✗	✗	✗	—	Não implementado
Pagamento PSP	✗	✗	✗	✗	Não implementado
Registro PaymentMethod	✓ close	✓ close	✓ enum	✓ colunas	Completo como enum
Estoque movimento	✗ mock	✗	⚠ campo+alerta	⚠ quantity	Parcial / incompleto
Dashboard loja	✓	✗	✓	✓ leituras	Completo
Dashboard master	✓	✗	✓ stores dashboard	✓	Parcial (gráfico mock)
Relatórios avançados	✗	✗	⚠ overview	—	Parcial
Configurações loja	⚠ mock não montado	⚠ tema local	✗	✗	Não implementado
Impressão térmica	✗	✓ Bluetooth	—	—	Completo local Android
Offline / sync	✗	✗	—	—	Não implementado
POST /order/new	✗	⚠ API morta	✗ deprecated	—	Abandonado
Garçom na web	⚠ placeholder	—	—	—	Não operacional

5. Diagramas Mermaid restantes

Autenticação

```

flowchart LR
    U[Usuário] --> W[Web LoginForm]
    U --> A[Android LoginActivity]
    W --> L[POST /api/auth/login]
    A --> L
    L --> AS[AuthService]
    AS --> DB[(tb_user)]
    AS --> JWT[JwtTokenProvider]
    JWT --> W
    JWT --> A
    W --> LS[localStorage]
    A --> DS[DataStore Proto]

```

Multi-tenant

```

flowchart TB
    User[User.store_id]
    F[JwtAuthenticationFilter]
    TC[TenantContext]
    AS[StoreHibernateFilterAspect]
    Q["SQL store_id = :storeId"]

    User --> F --> TC --> AS --> Q

```

Web → Backend

```

flowchart LR
    Login[Login] --> Auth[POST /api/auth/login]

```

```
Master[MasterDashboard] --> Stores[ /api/stores ]
Emp[Employees] --> E[/api/employees]
Menu[Menu] --> Cat[/category /product]
Att[Atendimento] --> T[/tables /comandas]
Att --> Acc[GET/POST contas]
Kit[Kitchen] --> K[/kitchen]
Dash[DashboardGerencial] --> D[/dashboard/overview]
```

Android → Backend

```
flowchart LR
    L[LoginActivity] --> Auth[POST api/auth/login]
    H[HomeFragment] --> Home[/home/options/all /home/shift]
    T[TablesFragment] --> TA[GET tables/all]
    C[ComandasFragment] --> CA[GET comandas/all]
    Cart[CartViewModel] --> PO[POST tables/id/orders]
    GO[GroupOrderViewModel] --> CL[POST close]
    GO --> KS[PATCH kitchen status]
    OD[OrderDetailsViewModel] --> CTR[POST counter/orders/id/close]
```

6. Índice da pasta

Doc	Tema
01-system-overview.md	Arquitetura geral
02-backend-architecture.md	Spring
03-web-architecture.md	React
04-android-architecture.md	Kotlin
05-authentication-flow.md	Login
06-multitenancy-flow.md	storeId
07-order-flow.md	Pedido
08-table-flow.md	Mesa
09-command-flow.md	Comanda
10-counter-flow.md	Balcão
11-kds-flow.md	Cozinha
12-payment-flow.md	Pagamento
13-inventory-flow.md	Estoque
14-api-map.md	Endpoints
15-database-model.md	Tabelas
16-integration-map.md	Dependências
17-problems-and-gaps.md	Gaps

Nenhum código de aplicação foi alterado nesta análise. Apenas estes arquivos de documentação foram criados.